King Saud University
College of Applied Studies and Community Services
Department of Computer Science and Engineering
Bachelor's Degree in Applied Computing – Cybersecurity Track

Protecting Smart Cameras from Cyber Attacks

Project Submitted in Fulfillment of the CYB 496 Course requirements.

By:

Student name	***ID***
Alaa Majed Batais	
Besan Al Habardi	
Lujain Harbi	
Ghala Al-Qahtani	
Rawan Al Qahtani	
Rana Al Harbi	

Supervised by:

Nada Al-Hirabi

Academic Year 2025-2026

Declaration

We are cyber security students being members of final year project group number one, declare that this report contains only work completed by members of our group except for information obtained legitimately from literature, company or university sources. All information from these other sources has been duly referenced and acknowledged by the University Policy on Plagiarism.

Furthermore, we declare that in completing the project, the individual group members had the following responsibilities and contributed in the following proportions to the outcomes of the project:

Student ID	Responsibilities	Contribution %	Signature
Alaa Batais	1.1 motivation	100%	
	2.4 Security systems and mitigation controls	50%	
	3.4 Security structure and defense systems	100%	
	4.9 System Interfaces	100%	
	4.10 Summary	100%	
	5.1 Proposed Solution	100%	
	6.7 protection phase	30%	
	7.2 MITM & network sniffing testting	100%	
Ghala Al-Qahtani	1.5 Project scope	100%	
	2.2 IOT environments with smart camera	100%	
	IP camera valnerabilities	100%	
	4.6 Functional Requirement (FRs)	100%	
	4.7 Non- Functional Requirements (NFRs)	100%	

	5.4 Procedures	100%	
	6.7 protection phase	40%	
	6.8 security monitoring dashboard	100%	
	7.3 RTSP Attack Testing	100%	
Rawan Al Qahtani	1 Introduction	100%	
	2.4 Security systems and mitigation controls	50%	
	3.3 Privacy threats and ethics impact	100 %	
	4.2 System Perspective	100%	
	4.3 Requirements Elicitation Techniques.	100%	
	5.6 Summary	100%	
	6.1 lab setup & environment	100%	
	6.6 attacks phase	30%	
	8.2 Future work	100%	
Besan Al Habardi	1.3 State of the art and research gap	100%	
	2.5 Ethics and privacy issues Detection and mitigation strategies	100%	
	4.4 System Requirement	100%	
	4.8 Smart Camera Security – Entity Relationship Diagram (ERD)	50%	
	5.2 Network Protocols	100%	
	6.2 web dashboard development	100%	
	6.4 network isolation and security testing environments	100%	
	6.6 attacks phase	30%	
Lujain Harbi	1.6 Report structure	100%	
	2.1 Introduction	100%	
	3.1 Overview	100%	
	4.1 Introduction	100%	

	4.8 Smart Camera Security – Entity Relationship Diagram (ERD) 5.3 Methodology 6.7 protection phase 7.1 brute forces attack testing	50% 100% 30% 100%	
Rana Al Harbi	1.2 Problem definition 1.4 Aims and objective 2.3 Typical cyber attacks on smart cameras 3.6 Chapter summary 4.5 Use case Diagram 6.3 attacker platform (kali linux) 6.5 reconnaissance 6.6 attacks phase 8.1 Conclusion	100% 100% 100% 100% 100% 100% 100% 40% 100%	

Acknowledgment

At the beginning, we would like to extend our warm thankfulness to Nada Al Hirabi, our academic supervisor, whose guidance, encouragement, and critiques have been instrumental in moving this study forward. The knowledge and wisdom she has in the field of leadership and organizational studies also helped a lot in structuring this study and her support throughout the whole process was greatly valued.

We would also wish to appreciate the faculty and staff at King Saud University /Computer Science and Engineering, whose efforts toward excellence in academics and research have made this study possible. This has helped us in establishing a dynamic and intellectually stimulating environment that enhanced our academic growth.

We also would like to express our gratitude to our colleagues and peers who have given encouragement, critical thinking, as well as constructive discussions throughout the time of this study. They contributed to improving the scope and quality of the study.

A little bit more personally, we are extremely thankful to our families and the people who are close to us, who supported us throughout this time and were patient and very understanding of our situation. They believed in what we aspired to do, and this has been quite inspiring to us.

Abstract

With the increasing use of smart cameras and their adoption in both home and workplace environments, they have become more vulnerable to cyberattacks due to weak authentication mechanisms, insecure communication protocols, and weak network configurations. To evaluate these risks, this project presents a practical security evaluation and protection framework for smart camera systems using a Raspberry Pi-based testbed environment. Several attacks, including brute-force attacks, network sniffing, RTSP exploitation, and Man-in-the-Middle (MITM), were simulated using tools such as Wireshark, Nmap, Hydra, and Bettercap. To improve system security, multiple protection mechanisms were implemented, including HTTPS encryption, Multi-Factor Authentication (MFA), account lockout protection, firewall rules, RTSP protection, and real-time email alerts. The results demonstrated that the implemented defenses successfully reduced unauthorized access attempts and improved the overall security posture of the smart camera system. Additionally, a security monitoring dashboard was developed to visualize security events and protection status.

Keywords:

Smart Camera Security; Internet of Things (IoT); Raspberry Pi; RTSP; Cybersecurity; Wireshark; Nmap

Abstract (in Arabic)

مع الانتشار المتزايد للكاميرات الذكية واستخدامها في المنازل وبيئات العمل، أصبحت أكثر عرضة للهجمات السيبرانية بسبب ضعف وسائل المصادقة، وبروتوكولات الاتصال غير الآمنة، وضعف إعدادات الشبكة. ولتقييم هذه المخاطر، يقدم هذا المشروع إطارًا عمليًا لتقييم وحماية أنظمة الكاميرات الذكية باستخدام بيئة اختبار قائمة على Raspberry Pi لمحاكاة سيناريوهات واقعية. وتم تنفيذ عدة هجمات، منها هجمة القوة الغاشمة(Brute Force) ، واعتراض حركة الشبكة، واستغلال بروتوكولRTSP ، وهجمة الرجل في المنتصف(MITM) ، باستخدام أدوات مثل Wireshark و Nmap و Hydra و Bettercap.

ولتعزيز أمن النظام، تم تطبيق عدة وسائل حماية، تشمل تشفيرHTTPS ، والمصادقة متعددة العوامل(MFA) ، وآلية قفل الحساب، وقواعد الجدار الناري، وحمايةRTSP ، والتنبيهات الفورية عبر البريد الإلكتروني. وأظهرت النتائج نجاح وسائل الحماية المطبقة في تقليل محاولات الوصول غير المصرح بها وتحسين المستوى الأمني العام لنظام الكاميرات الذكية. كما تم تطوير لوحة مراقبة أمنية لعرض الأحداث الأمنية وحالة الحماية.

List of Abbreviations

Abbreviation	Full Term	Description
IoT	Internet of Things	Network of interconnected devices that communicate and exchange data over the internet.
IP	Internet Protocol	Communication protocol used for addressing and routing packets between devices.
HTTP	Hypertext Transfer Protocol	Protocol for transferring web content and communication between clients and servers.
TCP	Transmission Control Protocol	Reliable transport protocol used for connection-oriented communication.
UDP	User Datagram Protocol	Lightweight, connectionless transport protocol used for fast data transmission.
RTSP	Real-Time Streaming Protocol	Protocol used for controlling streaming media between clients and servers.
DDoS	Distributed Denial of Service	Cyberattack that overwhelms a target device or network with massive traffic.
MITM	Man-in-the-Middle	Attack where an attacker intercepts communication between two devices.
MFA	Multi-Factor Authentication	Security method that requires more than one form of verification to access.
VLC	VideoLAN Client	Media player used for streaming and viewing multimedia content.
LAN	Local Area Network	Private network connecting devices within a limited area, such as a home or lab.

Table of Contents

Declaration ..... **ii**

Acknowledgment ..... **v**

Abstract ..... **vi**

Abstract (in Arabic) ..... **vi**

List of Abbreviations ..... **vii**

List of Figures ..... **xii**

List of Tables ..... **xv**

1. Introduction **1**

1.1 Introduction 2

1.2 Motivation 3

1.3 Problem Definition 4

1.4 State of the Art and Research Gap 4

1.5 Aims and Objectives 5

1.6 Project Scope 6

1.7 Report Structure 7

2. Background **9**

Background **10**

2.1 Introduction 10

2.2 IoT environments with Smart Cameras. 12

2.3 Typical Cyber attacks on Smart Cameras 13

2.4 Security Systems and mitigation controls 14

2.5 Ethical and Privacy Issues 18

3. Literature Review **19**

Literature Review **20**

3.1 Overview 20

3.2 IP Camera Vulnerabilities 21

3.3 Privacy Threats and Ethical Impact 22

3.4 Security Structures and Defense Systems 23

3.5 Detection and Mitigation Strategy 24

3.6 Chapter Summary 25

4. System Analysis **28**

System Analysis **29**

4.1 Introduction 29

4.2 System Perspective 30

4.3 Requirements Elicitation Techniques. 31

4.4 System Requirement 32

4.5 Use case Diagram 34

4.6 Functional Requirement (FRs) 39

4.7 Non- Functional Requirements (NFRs) 41

4.8 System Interfaces 42

4.9 Summary 43

5. Proposed Solution **45**

Proposed Solution **46**

5.1 Proposed Solution 46

5.2 Network Protocols 47

5.3 Methodology 48

5.4 Procedures 50

5.5 User Interface Design 51

5.6 Summary 52

6. Implementation 53

Implementation 54

6.1 Lab Setup & Environment 54

6.1.1 Target Device Configuration (Raspberry Pi) 54

6.2 Web Dashboard Development: 56

6.3 Attacker Platform (Kali Linux) 59

6.4 Network Isolation and Secure Testing Environment 59

6.5 Reconnaissance 60

6.6 Attacks Phase 62

6.6.1 Brute Force Attack 62

6.6.2 RTSP Brute Force Attack 63

6.6.3 Network Sniffing Attack 65

6.6.4 Man-in-the-Middle (MITM) Attack 66

6.7 Protection Phase 68

6.7.1 Brute Force Attack Protection 68

6.7.2 Man-in-the-Middle (MITM) & Network Sniffing Protection 70

6.7.3 RTSP Attack Protection 73

6.7.4 Email Notification Configuration 76

6.8 Security Monitoring Dashboard 77

7. Testing 78

7.1 Brute Force Attack Testing 79

7.1.1 Brute Force Attack Before Protection 79

7.1.2 Account Lockout After Multiple Failed Attempts 79

7.1.3 Multi-Factor Authentication Verification 80

7.1.4 Real-Time Email Alert Notification 81

7.1.5 Testing Result 82

7.2 MITM & Network Sniffing Testing 83

7.2.1 Network Sniffing Before HTTPS Protection 83

7.2.2 Network Sniffing After HTTPS Protection 84

7.2.3 MITM Attack Before HTTPS Protection 85

7.2.4 MITM Attack After HTTPS Protection 85
7.2.5 HTTPS Protection Status Verification 86
7.2.6 Real-Time Email Alert Notification 87
7.2.7 Testing Result 88
7.3 RTSP Attack Testing 89
7.3.1 RTSP Access Before Protection 89
7.3.2 RTSP Protection Enabled 90
7.3.3 Real-Time Email Alert Notification 91
7.3.4 Testing Result 91
8. Conclusion and Future Work 93
8.1 Conclusion 94
8.2 Future Work 94
8.2.1 Artificial Intelligence and Anomaly Detection 94
8.2.2 Advanced Authentication and Access Control 95
8.2.3 Centralized Monitoring and Scalability 95
References 96
Appendix 99

List of Figures

Figure 1 Types of Cyber Attacks Targeting IoT and Smart Camera Systems [1]. 2
Figure 2 Examples of Physical/Perception Layer Security Attacks in IoT Devices[14] . 10
Figure 3 Common Threats [23].....15
Figure 4 Common types of attacks [23].....16
Figure 5 Smart camera security testing architecture diagram.....31
Figure 6 Smart camera Security testing Use case Diagram. 34
Figure 7 System Modeling diagram.....42
Figure 8 Proposed system flowchart.....43
Figure 9 Methodology flow chart. 49
Figure 10 Homepage User Interface showing the main project functions.....51
Figure 11 Lab hardware setup including the Raspberry Pi, camera module, and the dedicated router.....54
Figure 12 MediaMTX terminal output confirming the server has loaded the configuration, started the camera stream, and opened the RTSP listener on port 554.....55
Figure 13 MediaMTX configuration file (mediamtx.yml) showing the manual setup of basic authentication (username: admin, password: 12345) to require credentials for RTSP stream access.....56
Figure 14 The web management authentication interface developed for the Smart Camera Dashboard. 57
Figure 15 The System Settings interface of the Smart Camera Dashboard, featuring password management and device information. 58
Figure 16 The frontend user interface showing the live embedded camera feed within the management dashboard.....58
Figure 17 Code snippet showing the iframe integration for the live camera stream. 59
Figure 18 Nmap scan results showing open services and exposed ports on the Raspberry Pi smart camera system.....60
Figure 19 The web interface was accessed via a browser, where a login page was observed. 61
Figure 20 Attempt to access the RTSP stream using VLC without credentials. 61

Figure 21 Hydra tool executing a brute force attack on the login interface. 62
Figure 22 Successful brute force result revealing valid credentials. 63
Figure 23 Python script used to perform the RTSP brute force attack. 64
Figure 24 Successful RTSP brute force result revealing valid credentials. 64
Figure 25 Captured HTTP POST request showing username and password in plain text. 65
Figure 26 Captured network traffic showing video data transmitted over HTTP. 66
Figure 27 Bettercap performing ARP spoofing and traffic interception. 67
Figure 28 Captured HTTP request during the MITM attack. 67
Figure 29 Security Monitoring Dashboard displaying protection status and recent security alerts. 77
Figure 30 Successful brute-force attack before enabling authentication protection. 79
Figure 31 Account lockout mechanism activated after detecting multiple failed login attempts. 80
Figure 32 Email-based MFA verification code sent after successful credential validation. 81
Figure 33 Real-time email alert generated after detecting a brute-force attack attempt. ... 82
Figure 34 Captured HTTP credentials and unencrypted network traffic before enabling HTTPS protection. 84
Figure 35 Encrypted TLS traffic after enabling HTTPS protection. 84
Figure 36 Bettercap intercepting unencrypted HTTP communication traffic before enabling HTTPS protection. 85
Figure 37 HTTPS protection enabled during MITM monitoring showing encrypted communication traffic. 86
Figure 38 Dashboard showing HTTPS protection disabled. 87
Figure 39 Dashboard showing HTTPS protection enabled. 87
Figure 40 Real-time email alert generated after detecting insecure HTTP communication. 88
Figure 41 Successful RTSP brute-force attack before enabling RTSP protection. 89
Figure 42 Successful RTSP stream access before applying RTSP protection mechanisms. 90
Figure 43 Blocked RTSP connection attempts after enabling RTSP protection. 90

Figure 44 Real-time email alert generated after detecting unauthorized RTSP access attempts. 91

List of Tables

Table 1 Summary of Reviewed Literature on IP Camera Security and Privacy. 26
Table 2 UC-01 — Configure Test Environment. 35
Table 3 UC-02 — Preform Reconnaissance Scan. 35
Table 4 UC-03 — Execute Controlled Exploitation. 36
Table 5 UC-04 — Analyze Network Traffic. 37
Table 6 UC-05 — Deploy Defensive Controls. 37
Table 7 UC-06 — Generate Security Report. 38
Table 8 UC-07 — Explore Security Report. 38
Table 9 Functional Requirements Summary. 40

Chapter
1. Introduction

1.1 Introduction

The Internet of Things (IoT) has transformed the contemporary connectivity, whereby billions of intelligent devices have been incorporated in households, industries, and cities. The IP-based smart cameras have been one of those devices that have become a critical part of smart environments and have played important roles of surveillance, automation, and remote monitoring. Nevertheless, this growth has also come with unprecedented cybersecurity threats which have jeopardized user privacy, integrity of their data and organizational sustainability. Adaptive learning frameworks to detect cyberattacks have become important in safeguarding such interconnected ecosystems as Kulkarni [1] highlights.

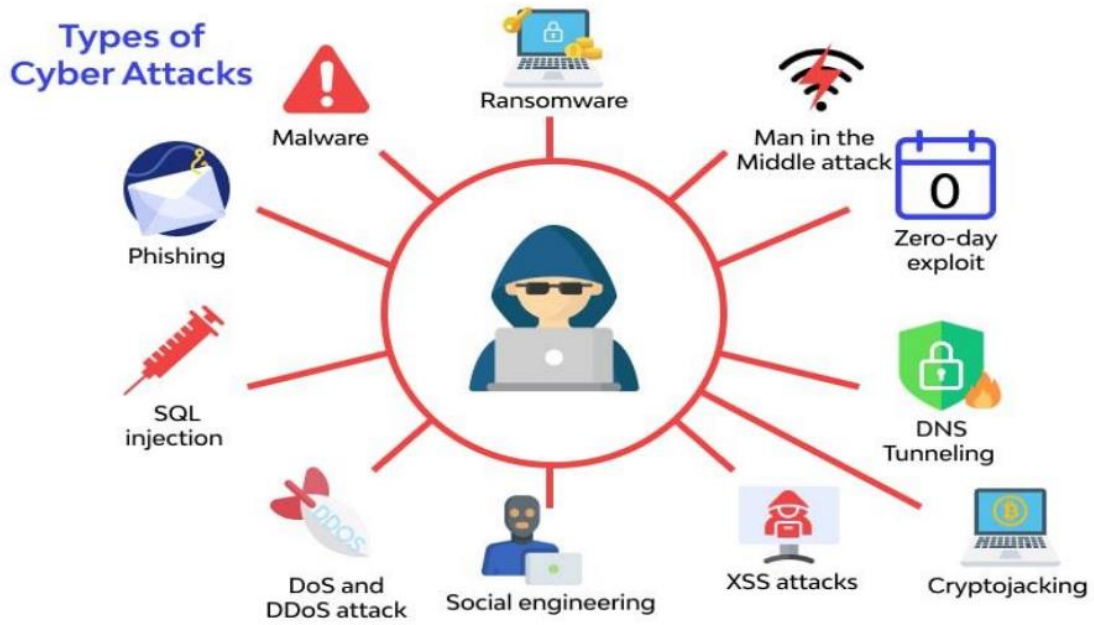


Figure 1 Types of Cyber Attacks Targeting IoT and Smart Camera Systems [1].

Figure 1 shows the key categories of cyberattacks that are often directed against Internet of Things (IoT) infrastructures, in this case, IP-based smart cameras. As illustrated, malware and ransomware attacks are used to exploit the vulnerabilities in the camera firmware and ineffective authentication systems that usually result in the encrypting of data or unauthorized access to the live feeds [1], [6], [12]. Phishing and social engineering techniques are used to trick users or administrators to disclose sensitive credentials or run harmful commands and enable the attacker to gain access to networks over the air [14]. SQL injection and cross-site scripting (XSS) are web-based attacks that utilize weak web interfaces and vulnerable camera dashboards to embed malicious code or steal user data

[5], [9], [10]. Such attacks on the network-level as denial of service (DoS) and distributed denial of service (DDoS) saturate devices or networks, turning the compromised cameras into botnet agents with which large-scale disruptions may be made [4], [7].

Several studies, such as Herodotou and Hao [5] and Kalbo et al. [6], found out that the attackers are capable of hacking into cameras to form stealthy surveillance links, do denial-of-service (DoS) attacks, or convert them into members of botnets like Mirai and Persirai. Innovations in detection methods have therefore risen--flow-based malware detection with federated learning [7] to internet of things penetration testing systems [11]. The evolution of IoT-related threats was also confirmed recently by Siwakoti et al. [12], Chee et al. [19], and Lefoane [20] and also provided opportunities to use AI-oriented intrusion detection as a means of improving the level of network protection.

Thus, the study will explore and examine the weaknesses of smart cameras, recreate real network attacks situation, and suggest a multi-layered defence model that incorporates technical hardening, behavioural scanning, and smart detection systems. The project aimed to make a contribution to the ever-increasing body of knowledge in the field of IoT forensics and preventive cybersecurity, contributing to the implementation of safe deployments of smart surveillance in the domestic and urban setting.

1.2 Motivation

With the growing use of Internet of Things (IoT)-enabled devices, especially IP-enabled smart cameras, the functioning of homes, businesses, and cities has been completely changed, providing real-time monitoring, automation and data-based decision-making. Though, such convenience has been costly: fast-growing attack surface that subjects users to privacy invasion, financial gain, and cyber-espionage. Smart cameras are frequently vulnerable to exploitation since Ling et al. [4] and Kalbo et al. [6] have noted that they are often configured using default credentials, have old firmware, and use weak communication protocols.

This study has been prompted by the fact that the number of cyberattacks using the IoT continues to grow at an alarming rate with smart cameras being commonly deployed to facilitate bigger network breaches or being incorporated into larger botnets like Mirai and Persirai [2], [7], [12]. Such attacks are not only disruptive, but also undermine the image

of users that has trusted surveillance systems that are supposed to protect them. Siwakoti et al. [12] and Chee et al. [19] have further shown that even compromised cameras have been used to commit financial crimes and cryptojacking in an effort to show the extent of the impact of poor IoT security.

Finally, the cultural reason of this project is the necessity to bridge the awareness and the practical defence gap and prove that active vulnerability analysis and relevant testing could be crucial to strengthen the cybersecurity stance of the smart camera implementation in a home and a workplace setting.

1.3 Problem Definition

The expansion of IP-based intelligent cameras in smart homes, businesses and infrastructures of the population has presented a frontier of issues of cybersecurity. Although these gadgets are meant to improve operational efficiency, surveillance and safety, they are becoming an ideal target of cyberattacks with its continuous connection, automatic default settings, and the little inbuilt security protocols. Researchers like Doughty et al. [2], and Alkassasbeh [8] note that most commercial IP cameras continue to use default credentials, unencrypted communication, and unsecured web interfaces, and are prone to attacks, including ARP poisoning, man-in-the-middle and remote code execution. This study, therefore, aims at defining, simulating, and testing the security vulnerabilities of IP cameras in controlled testbed environments, the vulnerability of these devices to attacks and the effectiveness of the various defense mechanisms- network configuration, and encryption - in preventing such attacks. It is aimed at providing quantifiable feedback on the methods of making IP camera deployments efficient, thereby ensuring the privacy of the information, the uninterrupted operation of the system, and confidence in the IoT-based surveillance apparatus.

1.4 State of the Art and Research Gap

The blistering increase in the Internet of Things (IoT) devices, specifically smart IP cameras, has transformed the sphere of home security, surveillance and automation. Nonetheless, this technological innovation has brought forth significant cybersecurity risks that are utilized by the enemies through malware attacks, botnet injections, and data

leakage. Z. Ling et al., (2018) state that end-to-end security of IoT devices is still in fractured state, and many manufacturers focus on cost-effectiveness and connectivity at the expense of high-quality encryption and authentication. In the same vein, N. Kalbo et al. (2020) also proved that IP-based video surveillance is vulnerable to brute-force attacks and unauthorized access due to misconfigured protocols (RTSP, and HTTP).

Nevertheless, even with such innovations, there is always a mismatch between academic suggestions and practical applications, mostly because of the shortage in testing conditions, absence of firmware-level access, and the absence of uniformity in device specifications in various manufacturers.

The contemporary art picture is such a fragmented landscape:

- There are detection frameworks, but they are not always real-time adaptable to changing malware in the IoT.
- Security models are suggested but seldom tested out by empirical network simulations using applications such as Wireshark or Nmap.
- There is little user-level security awareness because a majority of end-users do not know about weak passwords, open ports, and firmware update vulnerabilities (Alkassasbeh, 2021).

The identified gap in the research is that the practical, testbed-based vulnerability test does not exist that can be used to fill the gap between theoretical security models and measurable, real world implications. This project responds to that gap directly through carrying out experimental vulnerability analysis and also by designing a layered defense architecture of IP cameras. The methodology combines network-level scanning, malware behavior simulation and intrusion detection to generate a repeatable, evidence-based model to reinforce the security posture of the contemporary IoT surveillance systems.

1.5 Aims and Objectives

The primary focus of the given project is the protection of IP-based camera systems through systematic vulnerability analysis, simulated real-world cyberattacks, and an introduction of layered defense controls that can help these systems improve their security posture in the context of an isolated IoT testbed setup. The study aims to learn what kind of threats the smart cameras are subjected to and what are the vulnerabilities in the way these cameras

communicate and what can be done to implement the practical solutions that can be used to enhance cybersecurity resilience.

Objectives:

- To design and implement an isolated Raspberry Pi-based smart camera testbed.
- To simulate selected attacks, including brute-force, MITM, network sniffing, and RTSP unauthorized access.
- To analyze network traffic using tools such as Wireshark and Nmap.
- To implement defense mechanisms including HTTPS/TLS, MFA, account lockout, firewall rules, and email alerts.
- To evaluate the system before and after applying protection mechanisms.

Selecting these goals, the study will make contributions to the field of the IoT cybersecurity by reducing the gap between theoretical studies and practical defense mechanisms- to ensure that the smart camera systems would be functional and secure in an ever-connected world.

1.6 Project Scope

Project aims at evaluating a thorough vulnerability analysis and security improvement model of IP-based smart cameras in the Internet of Things (IoT) applications. It seeks to fill the void between theory and practice through a simulation of real-life attack situation and the possible defence measures within a controlled network environment. The scope will involve identification, analysis and mitigation of vulnerabilities that expose IP cameras to cyber threats in the form of brute-force attacks, open port attacks, and ARP poisoning. The research will utilize applications like Wireshark, and Nmap to capture and analyse live network traffic that will be used to identify possible signs of compromise and evaluate the activity of simulated attacks. The report will also discuss network, host and application level defences such as, encryption protocols, and intrusion Detection systems (IDS).

Finally, the research will also be extended to the design of a reproducible, layered, security framework of IoT surveillance devices that can offer quantifiable benefit in withstanding a cyberattack. The results of the project will equip the project with technical knowledge of cybersecurity practitioners and practical advice to the manufacturers and users on how to better secure smart camera systems against the changing threats in the digital ecosystem.

1.7 Report Structure

The report is divided into eight key chapters covering the background, literature review, system design, implementation, and security testing of the proposed smart camera security solution.

- Chapter 1 – Introduction
    - This chapter presents the background, motivation, and purpose of the research.
- Chapter 2 – Background
    - This chapter gives an overview of the IoT architecture, particularly the smart camera communication protocols, including HTTP, RTSP, and Telnet protocols.
- Chapter 3 – Literature Review
    - This chapter critically evaluates the recent scholarly work and industrial research connected to IoT vulnerability, camera security breaches, and malware functionality.
- Chapter 4 – System Analysis
    - This chapter provides a description of the system structure and design, including the hardware, software, tools (e.g., Wireshark, Nmap), and ethical considerations.
- Chapter 5 – Proposed Solution.
    - This chapter presents the proposed security testing solution and explains how it addresses the identified smart-camera vulnerabilities.

- Chapter 6 – Implementation
    - This chapter describes the implementation of the smart camera security system, including the Raspberry Pi setup, web dashboard development, RTSP streaming configuration, practical attack simulations, and security control implementation.

- Chapter 7 – Testing

- This chapter presents the testing and evaluation phase used to assess the effectiveness of the implemented security protections against the simulated attacks.

- Chapter 8 – Conclusion and Future Work
  - This chapter presents the project conclusion, summarizes the main findings, and discusses future research recommendations and possible enhancements.

Chapter 2. Background

Background

2.1 Introduction

The Internet of Things (IoT) is a networked digital environment of physical objects such as home sensors and industrial machines that can collect, exchange, and process data via the internet without involving a human operator [4]. The devices are powered by embedded systems, cloud platforms, and wireless networks to operate as a whole creating an intelligent infrastructure that allows automation, monitoring and data-driven decision-making [13]. Nonetheless, IoT is susceptible to various cybersecurity threats due to the identical connectivity that characterizes it. The hardware has scarce resources in most IoT devices such as cameras and sensors in the smart homes and as such, limits the use of advanced encryption and real-time threat detection [14].



Figure 2 Examples of Physical/Perception Layer Security Attacks in IoT Devices[14] .

The variety of security threats exhibited by the Internet of Things (IoT) ecosystem at the physical and perception yields is depicted in Figure 1. These layers are comprised of the

devices directly feel, gather and deal with the physical environment thus they are very vulnerable to attacks on the cyber- and hardware-level. Every interconnected device depicted in the diagram is a potential point of vulnerability that the attackers may use to undermine the integrity of the data, privacy of the users or the functionality of the systems. The IP camera is among the IoT components that are most susceptible to malicious actions, and many attacks focus on the malicious insertion of nodes, where the offenders achieve unauthorized access to live feeds or implant malformed firmware to compromise visual information. Likewise, smart meters are vulnerable to node tampering, which will allow attacks by hackers to alter energy use data or interfere with communication between meters and utility servers. Though the smart scale may appear harmless, it may fall victim to eavesdropping attack where the sensitive health or weight information sent through unencrypted networks is compromised by unauthorized parties.

Moreover, EV chargers are highly vulnerable to malware and software crashes that may terminate the process of charging or reveal personal details of the user. The smart thermostat, which is a common feature of the contemporary smart homes, can be hacked by malicious code injection, thereby leaving the attackers with the ability to take control of the household systems remotely. Similarly, intelligent vehicles are becoming more vulnerable to the insecurity of unauthorized access, in which case, with the help of poor authentication or unsatisfactory Wi-Fi networks, attackers can interfere with vehicle parameters or surveillance.

Other appliances like the garage door openers are also attacked through the social engineering method where the users are fooled into disclosing their access codes or remote access. In the meantime, drones are one more essential target on the physical layer because they can be used to hijack the navigation or threaten video transmission because of hardware exploitation.

Taken together, the above illustrations underline the fact that the perception layer of IoT represents not only a data collection frontier but also a main cybersecurity battlefield. The security of such systems involves the use of multi-layered defenses, such as effective encryption, regular updates of the firmware, and intrusion detectors. IoT infrastructures will be more resistant to attacks targeting it by reinforcing security at the perception layer or those that are a result of physical manipulation or exploitation of low-level devices.

The multiple challenge emerges because of the heterogeneous aspect of the IoT environments that do not have the standardized security. Certain gadgets produced by other vendors tend to utilize in-house standards, with interoperability and software support remaining unreliable [21]. These vulnerabilities are used by the attackers in malicious code injections, credential stuffing, or exploitation of open network ports [22]. The Mirai botnet attack, which is among the biggest IoT-based cyberattack, is a case example of how default passwords and weak authentication can make simple cameras into distributed denial of service (DDoS) weapons [19].

In addition, the increasing exponentially evolving IoT networks have exceeded the creation of the relevant cybersecurity models. Albalawi and Almaiah [22] underline that the quick growth of IoT provides a wide scope of attack, particularly when it is linked to cloud solutions that are operated in an unencrypted manner and transmit sensitive information. This poses the urgent requirement of end-to-end security models, the discussion of which is presented in [4] by Ling et al., who consider the encryption mechanisms, secure key exchange, and continuous monitoring to be the necessary ones.

To sum up, the IoT ecosystem is an opportunity and a threat. Although it allows automating data based on information, it is associated with weaknesses in authentication, firmware security, and interconnection, thus being the center of attention of cybercriminals. An effective security infrastructure that incorporates both device-level security, network segmentation, and anomaly detection is hence fundamental in ensuring that it is safely deployed in the domestic and industrial environment [15], [20].

2.2 IoT environments with Smart Cameras.

One of the most commonly used IoT devices is smart IP cameras because they are needed in surveillance, automation, and monitoring of smart cities [6], [23]. These are cameras with built-in image sensors, microcontrollers and network interfaces to record and transmit real-time video data either wirelessly or via wire. Most smart cameras are usually based on the communication protocols like HTTP, RTSP (Real-Time Streaming Protocol) and Telnet to control and stream capabilities [3], [10]. These protocols were however initially developed on the functionality aspect but not on security and the IP cameras can be easily accessed and compromise [8].

Research carried out by Kalbo et al. [6] and A. Alkassasbeh [8] indicated that more than 60 percent of commercially sold IP cameras are set up with default passwords, and thus can easily be attacked in a brute force attack. Further, poor encryption or unencrypted transmission of data subjects users to infringement of their privacy where a hacker can intercept live feeds. The unsecured Universal Plug and Play (UPnP) services are also broadening the risk of the easy finding and control of the remote devices without strong authentication [11].

Smart cameras are essential in video analytics, motion detection, and AI-based surveillance in the industrial and urban space. The installation of these systems into mutual networks however presents multi-layer vulnerabilities. Herodotou and Hao [5] argue that surveillance has been abused by the use of hidden or unauthorized cameras implying that the issue of privacy is a crucial aspect of their security concerns.

The camera becomes more complex to analyze visual information on the edge, which also introduces another dimension to the threat landscape. With the help of adversarial patches, adversaries can be used to mislead the object detection models as evidenced by Kwon and Kim [9], and it proves that even machine learning-powered cameras cannot withstand cyber manipulations.

Thus, the security of smart cameras should be considered as a single aspect that includes hardware hardening, encrypted protocols and access control. This involves frequent firmware updates, disabling of services that are not secure, and the use of multi-factor authentication. It is only under this multi-layered solution that IoT surveillance systems will be resistant to cyber and physical attacks [10], [16].

2.3 Typical Cyber attacks on Smart Cameras

Advanced cyberattacks are focused on smart cameras and attempting to exploit vulnerabilities in communication protocols, authentication or firmware [2], [7], [12]. The commonest threats are botnet recruitment, DoS attacks, guessing credentials brute force, and malware infiltration. The notorious Mirai virus uses a dictionary of default passwords to compromise devices that have open Telnet ports (23/2323) and turn them into attack nodes [19]. Doughty et al. [2] showed in the experiments of ARP poisoning that compromised cameras can redirect traffic, but such a camera can fully monitor the activity of users.

Equally, the Persirai malware also uses vulnerabilities in the TCP port 81 and the UpnP to gain access to smart cameras to execute scripts and report to the command and control servers [12]. These devices can be used to initiate UDP flooding attacks once they have been infected, which adds to the global level of DDoS attacks. According to Siwakoti et al. [12], even monetized IP cameras have been compromised by being turned into use in fraud online transactions or cryptocurrency mining, through cybercriminal infrastructure.

Recent studies show that the threat vectors are changing, and one of them is adversarial AI attacks, where the data are fed into the visual recognition systems by malicious parties to mislead them in recognition, as reported by Kwon and Kim [9]. According to Zhang et al. [7], the federated learning-based malware detection was introduced, however, the authors observed that even AI-based approaches fail to detect the zero-day vulnerabilities. In addition, El-Taj [16] notes that hackers use ineffective data encryption to interfere with the camera feeds resulting in serious privacy violations.

The overlapping of various threats at three levels, network, application, and hardware, requires a unified strategy in defence. These involve firewall protection, ciphers of communication protocols and detection of anomaly on the traffic [14], [20]. In the absence of such built-in measures, intelligent cameras will serve as entry-points to bigger IoT botnets. Therefore, the realization of these threat dynamics is core in creating an effective mitigation structure.

2.4 Security Systems and mitigation controls

It is important that smart cameras have a multi-layered security approach to safeguard them against cyber threats. The initial line of defense is powerful authentication tools- the replacement of factory set credentials with unique and complicated passwords and role-based access control implementation [14], [15]. Frequent updates on the firmware seal backdoors that hackers use. Also, intrusion detection systems (IDS) and Firewalls, as well as VLAN segmentation can be of great help in minimizing attack surface [20].

Prakash and Nambiar [10] came up with a privacy saving architecture that combines encrypted data transfer and secure data storage. According to Biondi et al. [11], lateral movement during the event of compromise is avoided through network isolation, in which the surveillance devices are on different subnets. Anomaly detection using machine

learning is also being used in modern systems to detect the abnormal patterns in the network [7].

Trusted Platform Modules (TPMs) and secure boot systems are hardware-based systems that offer protection against firmware integrity. At the software level, suspicious behavior can be identified early by continuously monitor the behavior of the system with Security Information and Event Management (SIEM) systems. This is enhanced by the fact that signature-based detection is used to detect known threats and behavioral analytics is used to detect unknown threats to help increase resilience to zero-day exploits [19].

Although the approaches enhance the safety of the devices, the awareness of the people using them is also a matter of concern. Many of these vulnerabilities, according to Alkassasbeh [8] and Herodotou [5], are a result of user negligence, including the existence of bad passwords and turned-off encryption. Therefore, technological interventions should be supported by cybersecurity education.

In total, mitigation mainly involves the enforcement of policies, automated defense system, and constant auditing. The AI-based threat intelligence must be incorporated into future solutions, which allow responding to the changing attacks in a more adaptive manner [27]. An IoT camera ecosystem that is truly secure should integrate components of cybersecurity both in the technical and human aspects.

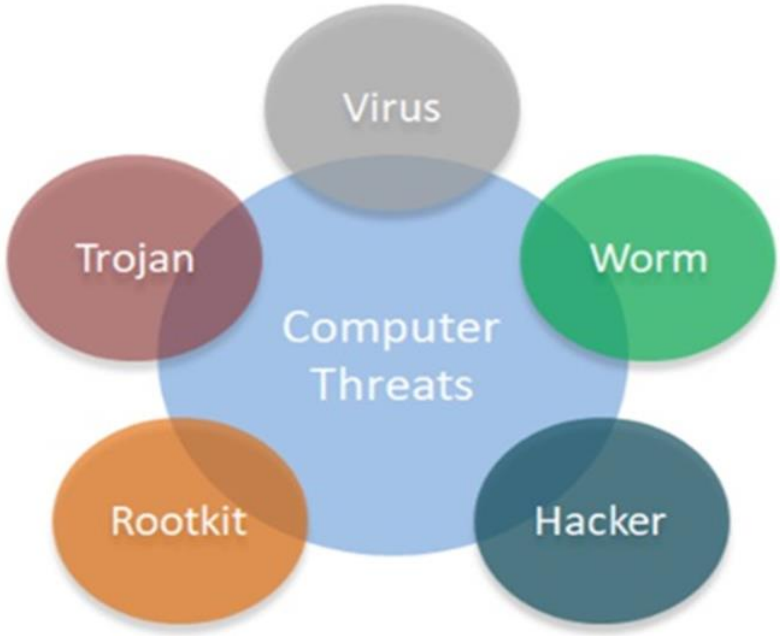


Figure 3 Common Threats [23].

Figure 3 demonstrates the key types of based threats that affect the security, privacy, and integrity of systems. The major circle with the name Computer Threats in it is the general idea of the evil activities that threaten the computer systems and networks. It has five major categories of threats surrounding it which are Virus, Worm, Trojan, Rootkit and Hacker and each threat has a different risk to the system operation and data.

Virus is a self-copying program which is attached on the legitimate files or programs after which it propagates when these files are launched. Worms, unlike viruses, are naturally automated and spread using network connection whereby they use gaps to infect several systems within a short period of time. To harm the user, a Trojan will appear as a harmless application to make the user install it, and in most cases, it will be used to gain unauthorized access or steal data. Rootkits allow attackers to have privileged access to a system without being detected by the detection tools, and make them especially dangerous and persistent. Finally, hackers use vulnerability in the system to steal information, interfere with services, or alter data because of personal or monetary benefits using manual or automated methods. Combined, these factors bring to the fore the different terrain of cybersecurity threats that could be directed at individuals, organizations, and critical infrastructure. These fundamental types are imperative in the application of sound defense measures, such as updating of systems, intrusion detection systems and user sensitization to reduce transforming threats.

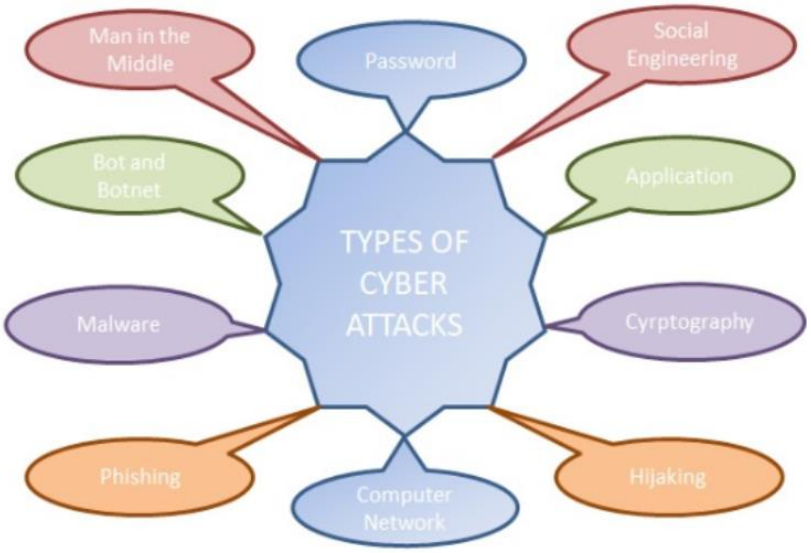


Figure 4 Common types of attacks [23].

The different groups of the most prevalent forms of cyber attacks, which affect digital systems, applications, and communication networks are listed in Figure 4. The focal point is the types of cyber attacks, which identifies the variety and changing essence of cyberattacks in digital age. The elements of the surrounding are used to depict the various attack vectors normally applied by cybercriminals to destroy confidentiality, integrity, and availability of data.

Password attacks: passwords are guessed or broken or stolen by brute force or phishing methods and used to gain unauthorized access into the system. Social engineering attacks tend to exploit the human behavior, fooling the users into giving up confidential information or taking actions which are harmful. Application attacks take advantage of bugs in computer programs or web applications and they are normally based on injection or buffer overflow attacks. Cryptography attacks use the encryption algorithm to get encrypted confidential information or create fake digital signatures to compromise secure communications.

Hijacking attacks e.g. session hijacking or DNS hijacking allow attackers to intercept or redirect user traffic and compromise connections. Computer networks attacks such as Denial of Service (DoS) and Distributed Denial of Service (DDoS) are attacks that are intended to saturate network resources and interrupt regular operations. The phishing attacks mislead users to disclose their credentials or download malicious programs by means of spoof emails or websites. Malware attacks are malicious programs that are installed on devices to infect and cause harm or steal information (virus, worms, Trojans). Bots and botnets attacks exploit vulnerable systems (bots) and remotely use them to make attacks on a large scale or deliver malware. Lastly, Man-in-the-Middle (MitM) attacks are those that intercept messages between two parties to eavesdrop or modify messages without being noticed.

All these types of attacks indicate that cyber threats are able to affect several levels of a digital ecosystem, including human users and network infrastructure. Research into the mechanisms of such attacks is critical to creating multi-layered defense steps, the use of secure authentication protocols, data transmission encryption, and increasing the awareness of the users to create resilient cybersecurity systems.

2.5 Ethical and Privacy Issues

In addition to the technical defense, ethical and privacy issues constitute a critical part of the IoT security. Surveillance cameras are IP cameras that are not built to protect the privacy of individuals (and may do the opposite), particularly when the information falls into the wrong hands or is stored without security guarantees [5], [16]. The emergence of secret cameras and illegal surveillance has brought up the issue of surveillance ethics as it has been brought to light by Herodotou and Hao [5].

Additionally, El-Taj [16] emphasizes the fact that the majority of smart cameras do not have clear policies on data-handling, allowing users to be exposed to privacy invasion by use of unencrypted cloud storage or sharing of data with third parties. In laws like the General Data Protection Regulation (GDPR), manufacturers must apply the principles of privacy-by-design and guarantee that the consent of the user and minimizing the data are prioritized [27].

Ethically, security researchers should also follow the responsible disclosure policies in the process of discovering vulnerabilities. A usage of vulnerabilities in research should be conducted under controlled settings without causing any damage to the users or organizations [15]. Vulnerability testing and ethical hacking are significant contributors to the development of the defensive knowledge without compromising trust towards technology [11].

There are new smart city projects that enhance these issues. Camera data within a networked setting can also demonstrate patterns of movement, behavior and even biometric identifiers [23], [25]. Hence, the end of user trust and safety over society require strong data governance structures and edge encryption models.

To sum up, privacy consideration in IP camera is not just a regulatory requirement, but it is an ethical requirement. Combining technical protection, open policies, and moral duty will make sure the advantages of surveillance technology could be achieved without sacrificing the personal rights [5], [16], [27].

Chapter
3. Literature Review

Literature Review

3.1 Overview

The emergence of the Internet of Things (IoT) has turned the conventional surveillance services into digital ecosystems that are interconnected. IP cameras are some of the devices, which are necessary to monitor homes and industries because it is less expensive, accessible, and comes with more features. But with the increasing connectivity, there is also increased cybersecurity vulnerability. The researchers have pointed IP cameras to be among the most used IoT devices due to default credentials, unpatched firmware, and poor encryption protocols [2], [6], [14]. Research like that of Ling et al. [4] and Kalbo et al. [6] indicate that these devices with continuous 24/7 operations are some of the most attractive points to cybercriminals in need of sustained network access.

Over the past years, researchers have been striving to know the attack surfaces of IP cameras. Doughty et al. [2] proved that Address Resolution Protocol (ARP) poisoning and Denial-of-Service (DoS) attacks could be used to disrupt the video feeds without any issue and expose real-time video. On the same note, Herodotou and Hao [5] pointed out that the vulnerability to hidden cameras may give the attackers access to the privacy areas without the need to physically enter them. With the growth of IoT ecosystems camera firmware and APIs have become more complex, there are several possible points of exploit [15].

According to recent articles by Siwakoti et al. [12], [18] it has also been revealed that even compromised IP cameras may be involved in financial fraud activities and Bhardwaj [17] advised that AI-based object detection systems may be blinded by adversarial patches making surveillance useless in case of an attack. These changing threats suggest that conventional security models cannot be effective.

Subsequently, researchers are currently interested in machine learning-powered systems of detection, blockchain-powered logging, and privacy-aware camera designs [10], [20], [27]. These strategies combine smart surveillance with immediate reaction whereby any anomalies are noticed before it is damaged. Although all this has been achieved, problems like ownership of data, regulatory loopholes, and human error still exist [8], [22].

On the whole, the literature highlights that although there has been a technological improvement of IP cameras, the cybersecurity practices have not been the same. This asymmetry poses an important dilemma: how to keep the level of usability and interconnectivity high without affecting the privacy and system integrity [1], [6], [19]. A

detailed study of vulnerabilities, privacy concerns, and defense mechanisms is therefore very important when developing future-proof surveillance networks.

3.2 IP Camera Vulnerabilities

The security vulnerability of IP cameras is based on their vulnerability analysis. Doughty et al. [2] have reported that most of the vulnerabilities are because of poor configuration, default login credentials and absence of firmware updates. Attackers can also use these weaknesses to carry out man-in-the-middle and packet sniffing attacks to steal video streams or take over command interfaces. Biondi et al. [11] conducted penetration testing on the best-selling IP camera models and disclosed the vulnerability to remote code execution (RCE) and buffer overflow vulnerabilities and showed that even widely used models do not have firmware that is hardened.

A study conducted by Kalbo et al. [6] on the attack surface of IP-based video surveillance systems revealed that there were various points of entry through the insecure RTSP and ONVIF protocols. Not only these weaknesses bring video feeds into the limelight, but also enable attackers to control the orientation and recording capabilities of cameras. In the same spirit, Alkassasbeh [8] observed that user oblivion is a key contributor - most devices can list themselves using public search engines like Shodan or Censys, which scrape the vulnerable IoT endpoints of the world.

Other studies indicate that the IP cameras can be recruited in botnets, like in the case of the Mirai and Persirai attacks [14], [19]. Having been compromised, these cameras are then used in DDoS or cryptojacking, which increases cyber threats in networks. Additionally, Bhardwaj [17] pointed out side-channel weaknesses, which allows attackers to determine the presence or activity of the user, even without having access to video feeds based on motion detection data.

El-Taj [16] built on these results and found defects in the data transmission channels such as the absence of the use of the encrypted web interface, the old web interface, and the handling of sessions. Interaction of technical flaws with user negligence leads to a layer of vulnerability model that can be exploited in sequence by the attackers such as reconnaissance to data exfiltration.

Thus, the literature recommends that the multi-layered protection such as excellent authentication procedures, encrypted streaming, automatic firmware patching, and network

partitioning are necessary to ensure the security of IP cameras. In the absence of these combined efforts, IP cameras are one of the most vulnerable and exploited elements of the IoT in the world [6], [8], [17].

3.3 Privacy Threats and Ethical Impact

In addition to the technical weaknesses, the privacy concerns of IP cameras are far-reaching. The devices are usually used in sensitive places such as homes, offices, hospitals and places of the common people and any unauthorized access directly infringes individual and organizational privacy. According to Herodotou and Hao [5], it is possible that the use of hidden surveillance technologies is abused to result into voyeurism or espionage with dire social and psychological effects. On the same note, Siwakoti et al. [12] showed that hacked IP cameras have been implicated in cyber extortion deals, where hackers are paid to avoid publishing personal videos to the populace.

El-Taj [16] noted that the threats to privacy are not only external but also internal since organizations or cloud vendors that archive footage without proper guidelines are also facing the threat. Most IP cameras use centralized servers hosted on the cloud, which are associated with the risks of data interception, unauthorized access of the third party, and exposure of the cross-border data. In addition, scholars like Alkassasbeh [8] raised the awareness of the user- lack of awareness: many users keep the devices on the Internet with their default settings and stream to the public IP addresses without knowing it.

Moreover, this has several ethical issues as far as the expansion of surveillance is concerned to the open environments. The line dividing the concept of public safety and the lack of privacy is being un-delimited as Sharma and Arya [25] note. Integration of AI-supported facial recognitions on cameras brings further aspect of ethical argument, as it may be used to profile or discriminate [24].

Bhardwaj [17] and Kwon and Kim [9] demonstrated that camera footage may be exploited to falsify truth and evidence by adversarial manipulation of video data, and this is a major challenge to the enforcement of the law and use of video footage in the courts. The risks that are presented demonstrate the urgent necessity of privacy-by-design system with the use of data anonymization, end-to-end encryption, and managed data retention policies.

The literature, in general, points to the development of policy reform, education of the people and ethical regulation to address privacy issues as not only technical innovation is

mandatory. The use of IP cameras should be utility-responsible, such that surveillance would not compromise the safety at the cost of trust and civil liberties [5], [12], [16], [25].

3.4 Security Structures and Defense Systems

There are several security frameworks that have been proposed by researchers to deal with IP camera vulnerabilities. Ling et al. [4] proposed a holistic IoT security architecture which incorporates the systems of encryption, authentication and intrusion detection to address network-level threats. Prakash and Nambiar [10] also developed a privacy-saving smart camera that process sensitive data locally and later send them to the cloud and has reduced chances of exposure to cyberattacks.

Biondi et al. [11] and Kalbo et al. [6] suggest adopting network segmentation and zero-trust networks to separate IoT equipment with the central business systems. These configurations constrain the horizontal traffic of malware and restrict access rights to preset policies. In the same manner, Siwakoti et al. [12], [18] emphasize the need to verify the firmware and perform authentication with a certificate to avoid a hijack of a device.

New studies are also pursuing the blockchain-based log system, which provides the traceability and non-repudiation of surveillance data [15], [19]. Devarakonda et al. [15] opined that decentralization of data storage would increase accountability and integrity is preserved. Additionally, Houichi et al. [27] presented an artificial intelligence (AI)-based Intrusion Detection System (IDS) specifically designed to operate on smart city infrastructures and which can use supervised learning models to identify anomalies.

Though, there is improvement, challenges still remain in the areas of scalability, cost of computation, and backward compatibility with older camera systems. Lefoane [20] conducted an overview of AI-based detection frameworks and resolved that deep learning improves accuracy but also complicates it and consumes energy.

The aggregate wisdom indicates a multi-layered defence model: such as hardened hardware design, protocol encryption, ongoing monitoring and self-adaptive threat intelligence. This has to be backed by regulatory systems that require security certification of IoT devices [19], [22]. Finally, the implementation of defence mechanisms requires technical resilience and human vigilance since advanced systems fail when users do not consider basic security hygiene [6], [11], [12].

3.5 Detection and Mitigation Strategy

The contemporary IP camera protection is becoming dependent on smart detection and proactive mitigation measures. Zhang et al. [7] have come up with a federated learning-based model that identifies zero-day IoT malware through training distributed nodes without exchanging sensitive data. The decentralized learning paradigm guarantees privacy and scalability of the large surveillance networks. On the same note, Albalawi and Almaiah [22] emphasized on machine learning classifiers to detect attack patterns through traffic anomalies and device behaviour analysis.

A survey of AI-based botnet detection was made by Lefoane [20], but based on hybrid models integrating statistical and neural methods to achieve increased accuracy. These systems will be able to identify infections at the initial stages, which will shorten the time spent offline and eliminate the spread of infections throughout the network. The study by Houichi et al. [27] presented an IDS system that is smart city-oriented, which dynamically changes detection thresholds according to the environmental conditions, enhancing the ability to adapt to the changing threats.

As El-Taj [16] and Prakash [10] state, mitigation measures comprise hardening of firmware, real-time encryption, and audit of user authentication. Biondi et al. [11] provided evidence of penetration testing that even simple network monitoring tools such as Wireshark may be used by the defenders to help identify ARP spoofing or even packet injection attempts at an early stage.

Moreover, Chee et al. [19] examined the development of IoT threats and suggested incorporating threat intelligence feeds with IDS/IPS systems to keep it updated. Defensive mechanisms should also be the same way up as cyber attackers use automation and AI since the defensive mechanisms need to improve to the level of predictive modelling and self-healing architecture.

To conclude, the existing literature indicates that the emphasis has changed to focus on proactive defence, as it is possible to use AI, automation, and distributed intelligence to identify, isolate, and neutralize threats prior to exploitation. Nevertheless, researchers concur that technology is not enough, security awareness and enforcement of policy alongside cross industry partnerships are also equally critical elements in the maintenance of sustainable IP camera security [7], [16], [19], [22], [27].

3.6 Chapter Summary

In this chapter, the literature review provided a thorough account of the literature review on the topic of vulnerabilities, privacy, and protection of IP cameras used in IoT environments. As it was revealed in the review, there is an increasing interconnectivity of IP cameras to smart home and surveillance systems, but at the same time, this has resulted in them being vulnerable to various cyber threats. The reviewed studies put together the idea that the most common points of attack are weaknesses in the authentication, firmware, and network settings that are being used by malicious actors [2], [6], [11].

Much of the literature underlines that the negligence of users, specifically the utilization of default passwords and obsolete firmwear is a major contributor to the maintenance of such vulnerabilities [8], [13]. Studies by Doughty et al. [2] and Kalbo et al. [6] have discovered the role of technical vulnerabilities such as open ports and weak encryption in the common network attacks, including ARP poisoning, DDoS, and man-in-the-middle (MITM) attacks. Such threats not only affect confidentiality and integrity, but also have a potential to lead to botnet perpetration on a large scale, such as the Mirai and Persirai attacks [14], [19].

Another theme of the chapter was the issue of privacy and the ethical implication. The researches such as those by Herodotou and Hao, [5], and El-Taj, [16] showed that IP cameras tend to record and archive sensitive information that can be misused to conduct surveillance, blackmail, or unauthorized profiling. The privacy issue can be further elaborated with the unauthorized access but also with the ethical use of the surveillance technologies and the issues about the cloud data management and the third-party exposure [12], [16], [25].

The literature also listed various defense frameworks that were aimed at alleviating these risks. Major suggestions are the end-to-end encryption, firmware authentication, network segmentation, and privacy-preserving camera designs [4], [10], [12]. The up-and-coming research also supports the combination of AI-driven intrusion detection and federated learning models to detect zero-day attacks instantly [7], [20], [27].

To sum up, the analyzed sources all remind an important lesson: even though much has already been done to discover the vulnerabilities and eliminate those of IP cameras, the sphere continues to struggle with providing real-time, adaptive, and scalable protection systems. Secure-by-design development, ethical surveillance policies and cross-

disciplinary models that incorporate cybersecurity, AI and privacy law should be prioritized in future studies to protect IP cameras in emerging IoT infrastructures [1], [6], [19], [24].

Table 1 Summary of Reviewed Literature on IP Camera Security and Privacy.

Theme	Authors	Key Contributions	Findings / Insights	Identified Research Gaps
IP Camera Vulnerabilities	Doughty et al. [2], Kalbo et al. [6], Biondi et al. [11], Bhardwaj [17]	Explored ARP poisoning, DoS, and firmware exploitation on IP cameras.	IP cameras suffer from weak authentication, default passwords, and insecure network protocols.	Lack of real-time vulnerability assessment tools for consumer-grade cameras.
Network & Protocol Weaknesses	Ling et al. [4], Alkassasbeh [8], Kumar et al. [13], Alladi et al. [14]	Studied attack surfaces in IoT networks, including insecure RTSP and ONVIF protocols.	Protocol-level weaknesses allow remote code execution and unauthorized access.	Limited encryption in streaming protocols; need for secure IoT communication standards.
Privacy & Ethical Challenges	Herodotou & Hao [5], El-Taj [16], Siwakoti et al. [12], Sharma & Arya [25]	Examined privacy violations and ethical concerns in camera surveillance.	Highlighted misuse of surveillance data for spying, extortion, and profiling.	Absence of standardized privacy frameworks and limited user awareness training.
Adversarial & AI-based Threats	Kwon & Kim [9], Bhardwaj [17], Chee et al. [19], Lefoane [20]	Identified adversarial attacks on AI-driven camera systems and botnet evolution.	Adversarial patches can disable object detection; AI models can be deceived.	Insufficient research on resilient AI and self-healing defense models.

Security Frameworks & Countermeasures	Prakash & Nambiar [10], Siwakoti et al. [18], Devarakonda et al. [15]	Proposed layered architectures, encryption, and blockchain logging.	Multi-layered defenses can mitigate many common threats.	Frameworks need scalability, automation, and interoperability for IoT systems.
Detection & Mitigation Approaches	Zhang et al. [7], Houichi et al. [27], Albalawi & Almaiah [22], Lefoane [20]	Developed machine learning and federated learning models for malware detection.	AI models improve zero-day malware detection without compromising privacy.	High computational cost and data privacy remain obstacles to real-world deployment.
Smart City & System Integration	Rehan [23], Houichi et al. [27], Sharma & Arya [25]	Discussed IP cameras as integral components of smart city infrastructure.	Integration of cameras with other IoT systems enhances safety but increases attack surfaces.	Need for unified cybersecurity policies across IoT ecosystems.
Ethical Hacking & Awareness	Javed [3], Alkassasbeh [8], Kulkarni [1]	Emphasized awareness, training, and ethical hacking practices.	Awareness and simulation-based testing reduce risk in deployment.	Lack of awareness campaigns for IoT device owners; insufficient hands-on ethical testing frameworks.

Chapter
4. System Analysis

System Analysis

4.1 Introduction

The proposed project is a controlled simulation framework designed to study and mitigate the security and privacy weaknesses of smart cameras that are vulnerable to network-based attacks. The solution addresses this problem by recreating, within safe and ethical laboratory environments, the lifecycle of common attack scenarios targeting smart cameras, with a particular focus on RTSP brute-force attacks and Man-in-the-Middle (MITM) techniques. These attacks exploit weak authentication mechanisms, exposed streaming ports, and improperly configured network services. The study replicates their behavior under realistic operational conditions in a virtualized environment to systematically model the stages of reconnaissance, credential compromise, and traffic interception. This approach enables the identification of effective methods to detect and disrupt these critical vulnerabilities before real-world damage occurs.

The solution will fill the research gap identified in the literature, where many studies analyzing IoT-related attacks are either theoretical or based on network traces without replicating the full attack lifecycle. Although a significant body of literature has focused on IoT malware such as Mirai and Persirai, limited empirical work has modeled real-world attack scenarios particularly those involving RTSP brute-force attacks and Man-in-the-Middle (MITM) techniques to understand device behavior at a granular level. This study addresses this gap by introducing a safe and practical Raspberry Pi-based emulation architecture that enables the evaluation of various defense mechanisms, such as firewalling and access control, under realistic attack conditions.

The proposed framework is directly aligned with the problem definition: consumer-grade IP cameras often lack robust security controls. These devices represent vulnerable entry points for unauthorized access and privacy breaches. The project adopts a systematic approach to evaluate the effectiveness of defensive countermeasures such as firewalling, credential hardening, and access control by measuring their impact on attack prevention and overall system security across different stages of the attack lifecycle.

Finally, this study contributes a validated experimental methodology that enhances understanding of IoT attack patterns and provides practical recommendations for manufacturers, network administrators, and policymakers aiming to improve the security of smart cameras and similar connected devices.

4.2 System Perspective

The smart camera (controlled via a Raspberry Pi) will be installed in a simple network environment in this project where it will be connected to a router that will be connected to a testing computer and the tools to scan and monitor it such as Nmap and Wireshark. The knowledge of these components interaction that the camera is able to communicate using the router, how data moves between the devices and how the testing tools are able to access the camera is the foundation of conducting the security evaluation. The process of conducting the security evaluation starts with reconnaissance to gain a deeper insight into the camera and the possible vulnerability in its services and communication protocols.

Once these weaknesses have been established, exploitation and simulated attacks are undertaken to observe the response of the camera when it is under the actual threat conditions. Security measures, firewall policies, video streaming via encrypted video, intrusion detection and strong authentication are applied to the camera or in its configuration interface. The attacks are then repeated on the same to find out whether the defenses put in place are effective in reducing or preventing the attacks. This configuration offers an entire workflow to the project: vulnerability discovery, vulnerability exploitation, defense mechanisms applied to the camera, and finally the evaluation of the effectiveness of each defense.

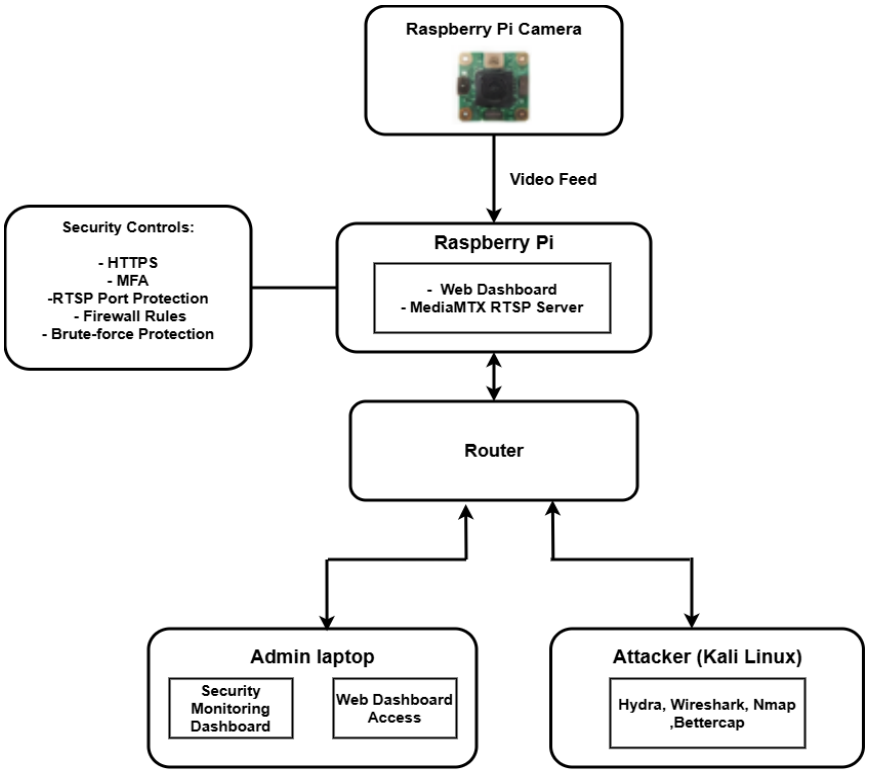


Figure 5 Smart camera security testing architecture diagram.

This figure illustrates the components and data flow within the smart camera security testing setup.

4.3 Requirements Elicitation Techniques.

The requirements of the project were identified using a structured approach that combines literature review, analytical evaluation of IoT security studies, and practical testing of available tools. The main objective of this phase was to define the requirements needed to create a realistic, controlled, and ethical environment for testing the security of a Raspberry Pi camera system against common network-based attacks.

The first step involved reviewing previous research on vulnerabilities and security risks in IP cameras and IoT devices. Through analyzing academic papers and case studies, common weaknesses were identified, including insecure communication protocols, weak

authentication mechanisms, and exposure to unauthorized access. These findings directly influenced the system requirements, such as the need for reconnaissance, vulnerability scanning, attack simulation, and traffic monitoring.

The second step focused on analyzing practical security testing approaches used in similar environments. This helped define how different components such as network monitoring tools, attack simulation scripts, and defensive mechanisms should interact to simulate real-world scenarios. The system was designed to support controlled testing of attacks such as brute-force attempts, RTSP access, network sniffing, and man-in-the-middle (MITM) attacks.

In addition, practical constraints were considered, including hardware availability, compatibility with penetration testing tools, and system performance. These factors influenced the non-functional requirements to ensure that the testing environment remains stable, efficient, and capable of handling activities such as scanning, attack simulation, and traffic analysis.

Finally, this process resulted in a clear set of system requirements, including hardware components, software tools, and security controls, all aligned with the project objectives and ethical testing standards.

4.4 System Requirement

The offered system was aimed at providing a regulated and safe setting to simulate and examine attack activity against Internet of Things (IoT) smart cameras. The system uses hardware and software components to fulfil this, and which together can be used to support network configuration, vulnerability assessment, traffic monitoring and controlled attack emulation. All components of the set up were chosen in accordance with their compatibility to the goals of the study and, also, their capability to provide safety, repeatability, and integrity of data during the entire experiment.

The system must have the following important components:

- Raspberry Pi Camera System (Primary Device)
    - A Raspberry Pi 4 equipped with a 5MP camera module. It serves as the primary target for security testing, acting as a programmable smart camera to simulate real-world IoT device behavior and network protocols.

- Network Router:
    - A dedicated network router is utilized to provide connectivity and manage communication between the system components. It ensures the isolation of the testing environment from external networks, keeping all experiments contained and secure.

- Testing and Monitoring Laptop:
    - A central laptop used to execute attacks, implement defense, and monitor the system. It runs Kali Linux via VirtualBox, utilizing tools like Nmap and Wireshark to evaluate vulnerabilities and the effectiveness of security measures.
- Security Controls and Defensive mechanisms:
    - To demonstrate a layered defense strategy, the experimental environment incorporates several security controls, including: firewall policies, encryption procedures (e.g., HTTPS), intrusion detection tools (IDS), and authentication systems. Such measures help to monitor the behavior of various defensive layers against simulated intrusion
- Attack Simulation Tools and Scripts:
    - Customized Python scripts and penetration testing techniques are used to simulate various cyberattacks, including brute-force attempts, port scanning, and unauthorized access scenarios. All experiments are conducted within a controlled and ethical environment to ensure safe security evaluation without affecting real-world systems.

Overall, this combination of components creates a controlled and ethically designed environment for evaluating the security vulnerabilities of IoT devices. The system supports both offensive (attack simulation) and defensive (protection and mitigation) procedures, enabling comprehensive evaluation of smart camera resilience within a controlled experimental environment.

4.5 Use case Diagram

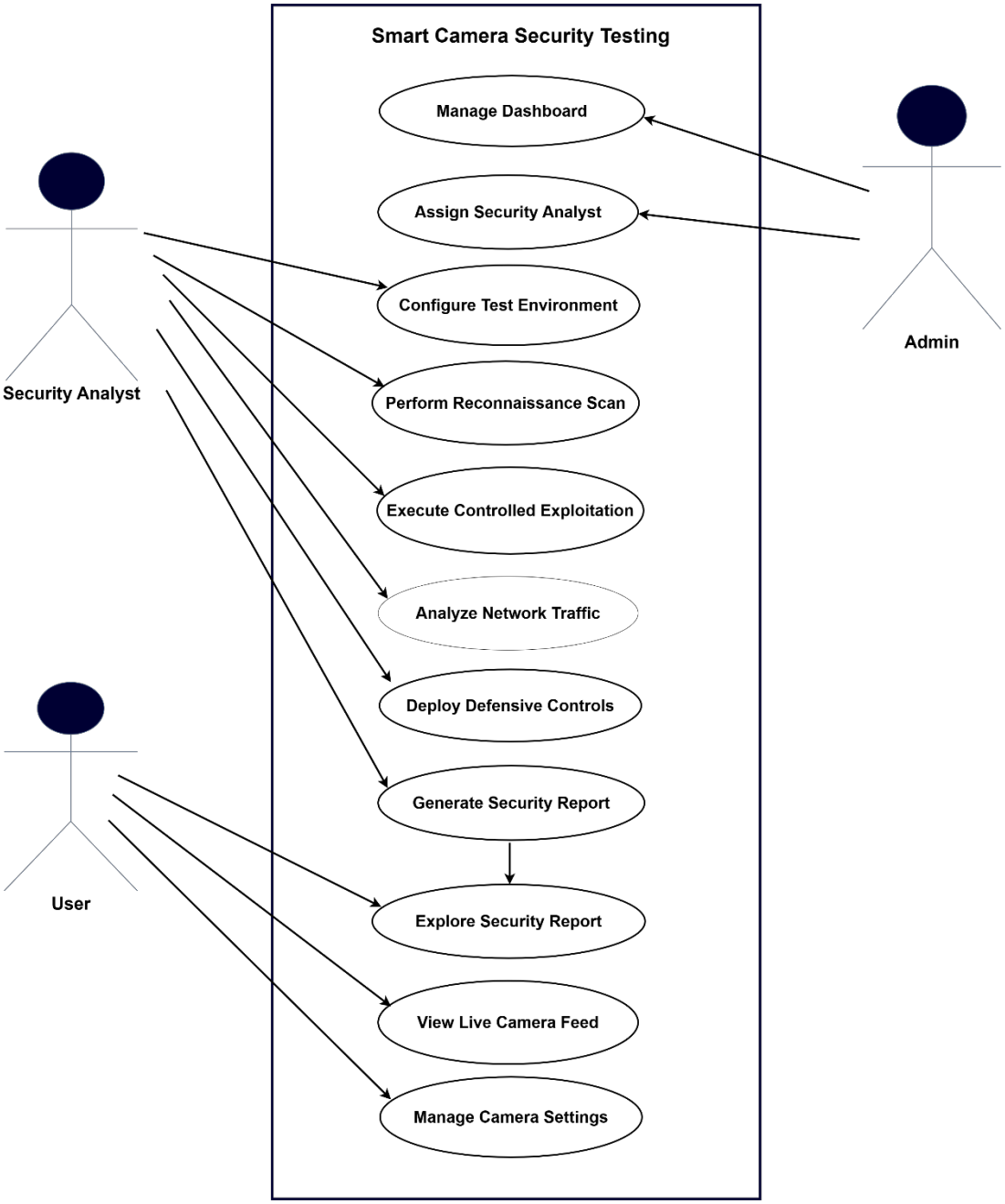


Figure 6 Smart camera Security testing Use case Diagram.

The Smart Camera Security Testing System will recreate, observe, and assess the dynamics of IoT cameras during possible cyberattacks and in a controlled research setting that is ethical. The use case diagram of the system demonstrates how the actors (Security Analyst, User, Admin) interact with functional modules of the testing structure.

Table 2 UC-01 — Configure Test Environment.

Field	Description
Use Case Number	UC-01
Use Case Name	Configure Test Environment
Description	The Security Analyst sets up and configures a controlled isolated test environment to prepare the smart camera for assessment. This includes connecting the camera to the isolated network, preparing required tools, configuring network components, and ensuring the environment is securely isolated from external access.
Actor	Security Analyst
Goal	To prepare a secure and functional environment that allows safe scanning, exploitation, and traffic analysis without affecting external networks or devices.
Precondition	The smart camera and network resources are available, powered on, and ready for integration into the test setup.
Postcondition	The test environment is fully configured, isolated, and ready for subsequent security testing phases.

Table 3 UC-02 — Preform Reconnaissance Scan.

Field	Description
Use Case Number	UC-02
Use Case Name	Perform Reconnaissance Scan

Description	The Security Analyst performs a network scan on the smart camera to identify active ports, exposed services, and general network visibility using tools such as Nmap or Wireshark.
Actor	Security Analyst
Goal	To discover accessible services and determine potential attack surfaces for subsequent testing.
Precondition	The smart camera is powered on, reachable, and connected to the isolated test network.
Postcondition	A detailed scan report is generated showing open ports, running services, and detected device fingerprints.

Table 4 UC-03 — Execute Controlled Exploitation.

Field	Description
Use Case Number	UC-03
Use Case Name	Execute controlled Exploitation
Description	The Security Analyst performs controlled and ethical attack simulations such as brute-force login attempts, RTSP access testing, and unauthorized interception scenarios to evaluate system vulnerabilities.
Actor	Security Analyst
Goal	To validate whether the camera is vulnerable to common IoT exploitation techniques and identify weaknesses.
Precondition	Service enumeration has identified potential vulnerabilities or attack surfaces.
Postcondition	Each exploitation attempt is evaluated, and its outcome whether successful or unsuccessful is documented to assess the actual exploitability of the identified vulnerabilities.

Table 5 UC-04 — Analyze Network Traffic.

Field	Description
Use Case Number	UC-04
Use Case Name	Analysis Network Traffic
Description	The Analyst performs network sniffing using Wireshark during the attack to capture data packets and identify sensitive information transmitted in plaintext.
Actor	Security Analyst
Goal	To intercept live traffic and provide technical evidence of data leakage and lack of encryption within the system.
Precondition	Camera activity has been triggered through normal operation or exploitation attempts.
Postcondition	Traffic logs are captured, revealing exposed credentials or unencrypted data.

Table 6 UC-05 — Deploy Defensive Controls.

Field	Description
Use Case Number	UC-05
Use Case Name	Deploy Defensive Controls
Description	The Security Analyst applies security measures such as encryption (HTTPS), strong authentication (MFA), firewall rules, and IDS policies to harden the camera.
Actor	Security Analyst
Goal	To verify the effectiveness of defensive strategies against previously observed vulnerabilities and attack simulations.
Precondition	Previous security weaknesses have been identified through UC-01 to UC-04.

Postcondition	Defensive configurations are deployed and validated, and their effectiveness is documented.

Table 7 UC-06 — Generate Security Report.

Field	Description
Use Case Number	UC-06
Use Case Name	Generate Security Report
Description	The Security Analyst compiles all findings from scanning, enumeration, exploitation attempts, traffic analysis, and defensive validation into a security report.
Actor	Security Analyst
Goal	To produce a complete evaluation of the camera's security posture and provide mitigation recommendations.
Precondition	All testing phases have been completed.
Postcondition	A final security assessment report is produced containing findings, risks, and recommended controls.

Table 8 UC-07 — Explore Security Report.

Field	Description
Use Case Number	UC-07
Use Case Name	Explore Security Report
Description	The User accesses the dashboard to explore the generated security report, reviewing detailed findings, identified vulnerabilities, risk levels, and recommended controls in a structured format.
Actor	User

Goal	To allow the User to review the generated security report and explore findings, vulnerabilities, risks, and recommended controls through the dashboard interface.
Precondition	A security report has been successfully generated after completing all testing phases.
Postcondition	The User successfully views the structured security report through the dashboard.

4.6 Functional Requirement (FRs)

The system should offer the following functional capabilities, to aid the smart camera security assessment, attack emulation and defensive validation, namely:

- FR-01: Device Discovery
    - The system will identify the smart camera on the network and retrieve the IP address, MAC address and vendor fingerprint of the camera.
- FR-02: Port and Service Scanning
    - The system will perform comprehensive port scanning to identify open ports, active services, and potential protocol vulnerabilities.
- FR-03: Service Enumeration
    - The system will be able to gather service details, authentication requirements, and active network services to be further analyzed in terms of vulnerability.
- FR-04: Credential Testing
    - Controlled credential testing (e.g. default credentials, weak password checks) and ethical and safe brute-force automation shall be conducted in the system.
- FR-5: Attack Simulation.
    - The system will be able to simulate controlled attack scenarios such as brute-force attempts, RTSP stream interception, network sniffing, and MITM interception in a safe environment.
- FR-06: Network Traffic Capture

- The system will be able to monitor and capture network traffic created by the camera, such as inbound and outbound packets.

- FR-07: Traffic Analysis
  - The system will be able to analyze the captured packets to identify sensitive information transmitted in plaintext and evaluate the security of network communications.
- FR-08: Vulnerability Detection
  - The system will be able to utilize scan results and attack outputs to identify security vulnerabilities, such as weak configurations and unencrypted services.
- FR-09: Deployment of Defensive Control.
  - The system will be able to implement and test various security enhancements and authentication controls to mitigate identified risks and improve overall system protection.
- FR-10: Reporting
  - The system will be able to provide a security overview that summarizes scan results, detected vulnerabilities, and the current status of defensive measures.

Table 9 Functional Requirements Summary.

FR	Requirement	Implemented?	Tested in section	Evidence
FR-04	Credential Testing	Yes	6.6.1/ 6.6.2/ 7.1	Hydra/script result
FR-05	Attack Simulation	Yes	6.6 / 7.1–7.3	MITM / RTSP/ Network Sniffing/ Brute-force results
FR-06	Network Traffic Capture	Yes	7.2 /6.6.3	Wireshark capture
FR-07	Traffic Analysis	Yes	7.2	HTTP/TLS analysis

FR-09	Deployment of Defensive Control	Yes	6.7 / 7.1–7.3	HTTPS / firewall / MFA
FR-10	Reporting	Partially	6.8	Dashboard figures

4.7 Non- Functional Requirements (NFRs)

These requirements are qualities and constraints of the system which makes the system to be reliable, secure and efficient.

- NFR-01: Security
    - The attack simulations must be contained within the isolated lab environment and should not have any effect on external systems or the main network.
- NFR-02: Reliability
    - The scanning, simulation, and logging tools should maintain stable performance throughout the testing sessions to ensure accurate data collection.
    - The findings must be repeatable in different test sessions.
- NFR-03: Performance
    - The scans, logging, and analysis of the system should be efficient.
    - The traffic capture tools should be able to handle real-time data with minimal delay and losses.
- NFR-04: Usability
    - The tools and environment should be easy to use by a researcher without necessarily being knowledgeable in complex technical execution.
    - Documentation will be easy to understand, organized and readable.
- NFR-05: Scalability
    - The testing framework should be in a position to accommodate other devices (e.g., other camera models) and add other scanning / analysis tools without significant structural modification.
- NFR-06: Maintainability

- All the scripts, configurations, and logs should be easy to update, archive, and make changes.
- The system should be designed to support future updates and enhancements for further research.

- NFR-07: Ethical Compliance
  - All testing activities must be conducted within the isolated lab environment to ensure no impact on external networks.
  - No destructive payloads, or deployment of malware will be employed.

4.8 System Interfaces

This diagram provides an overview of the main components of our system and the key deliverables it generates.

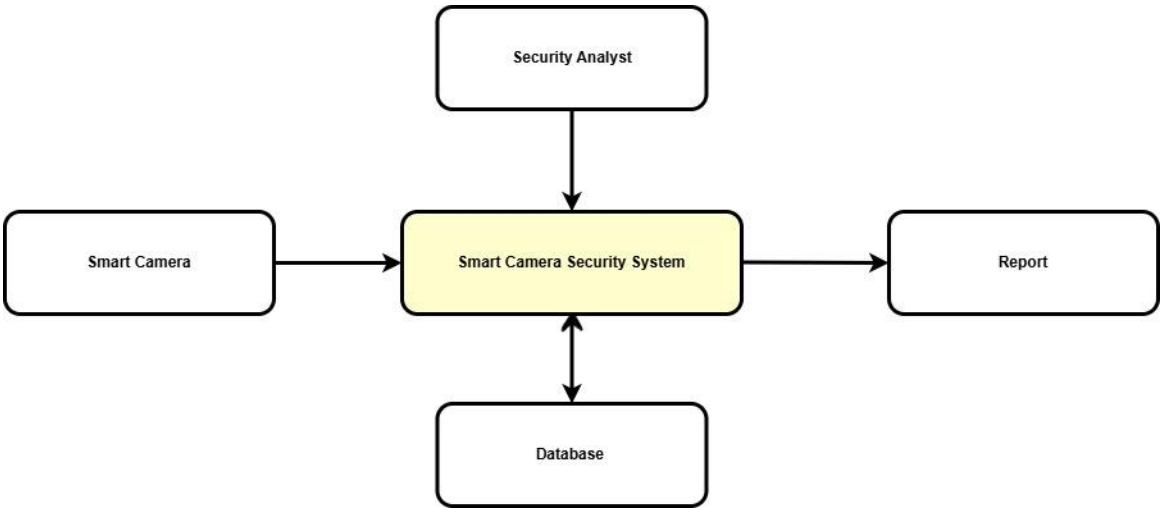


Figure 7 System Modeling diagram.

This diagram represents how the Security Analyst and the camera system interact with each other through the system vulnerabilities are identified, exploited, defended against, and re-tested to determine the improvement in security levels.

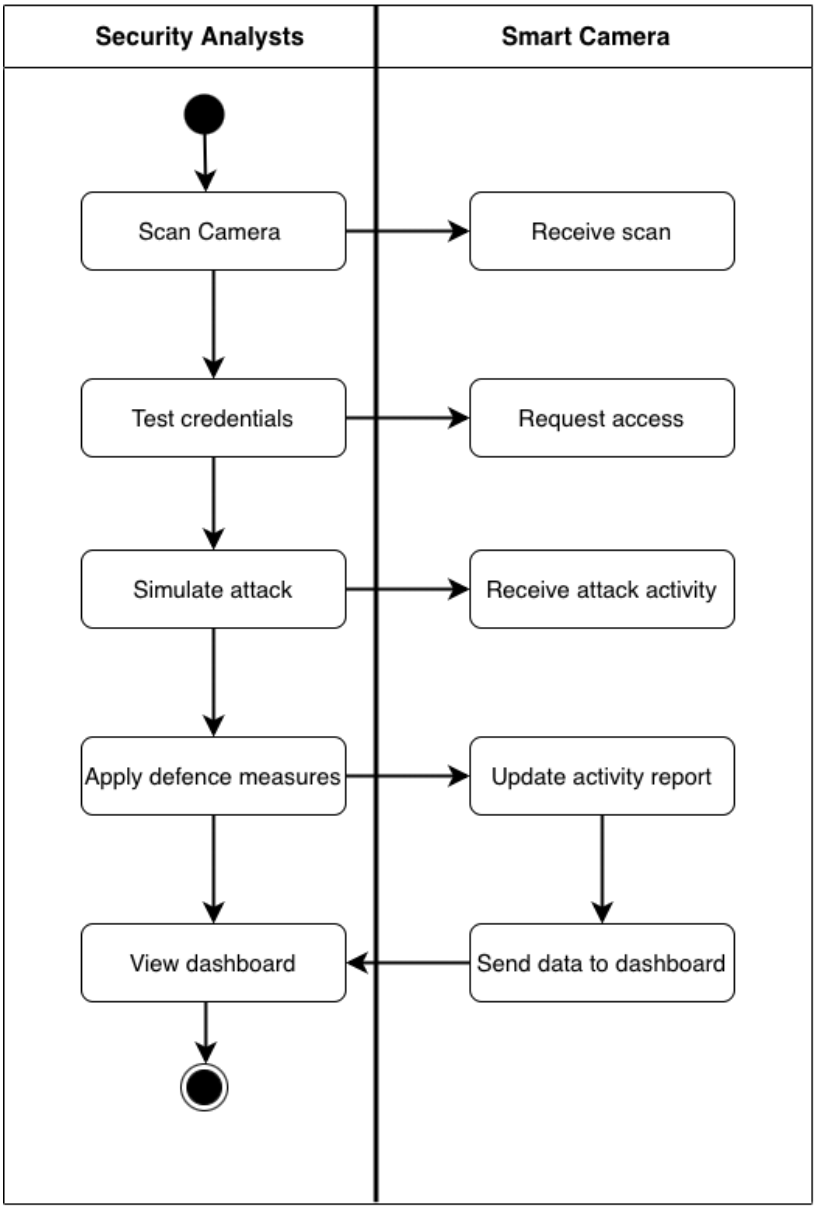


Figure 8 Proposed system flowchart.

4.9 Summary

This chapter presented the system analysis of the proposed Raspberry Pi camera security testing environment. It provided a detailed explanation of how the system was designed to support controlled and ethical security evaluation of a Raspberry Pi-based camera setup

within an isolated laboratory environment. The chapter clarified the relationship between the hardware components, the software tools, and the communication processes required to perform the testing scenarios.

The analysis covered the main workflow of the project, beginning with the setup of the testing environment and the identification of exposed services and possible vulnerabilities. It then explained how the system supports controlled attack simulation, including brute-force attempts, RTSP stream interception, network sniffing, and Man-in-the-Middle (MITM) analysis. In addition, the chapter showed how defensive controls can be introduced and evaluated to determine their effectiveness in improving system security.

Furthermore, this chapter defined the main system requirements, use cases, and both functional and non-functional requirements needed to achieve the project objectives. It also explained how the Raspberry Pi camera system interacts with the testing tools and how the generated data can be used to assess weaknesses and evaluate protection mechanisms. Overall, this chapter established the technical foundation for the implementation phase and provided a clear understanding of how the system behaves during both attack and defense scenarios.

Chapter
5. Proposed Solution

Proposed Solution

5.1 Proposed Solution

The proposed solution focuses on developing a structured, controlled, and repeatable security testing environment for a Raspberry Pi camera system. The purpose of this solution is to evaluate the security posture of the system by examining its exposure to realistic network-based threats and then measuring the effectiveness of different protection mechanisms. As discussed in the problem definition, camera systems connected to IoT environments are often vulnerable due to weak authentication, insecure communication protocols, service exposure, and insufficient protection of transmitted data. These weaknesses can allow attackers to gain unauthorized access, intercept sensitive information, and compromise the confidentiality of both user credentials and video streams.

To address these issues, the proposed solution establishes a realistic laboratory testbed based on a Raspberry Pi camera system within a safe and isolated environment. The testbed includes the target device, the network infrastructure, the web-based management interface, and the security testing tools required to perform controlled experiments. This setup allows the project to evaluate the system under different attack scenarios while maintaining ethical boundaries and avoiding any impact on external systems. The environment is specifically designed to support practical testing of vulnerabilities related to login security, RTSP access, unencrypted communication, and network-based interception.

The solution follows a complete workflow that includes reconnaissance, vulnerability identification, controlled attack execution, deployment of defensive mechanisms, and re-evaluation of the system after protection is applied. During the testing phase, realistic attack scenarios are carried out, including brute-force attempts against authentication interfaces, RTSP access testing, network sniffing using Wireshark, and Man-in-the-Middle (MITM) interception using Bettercap. Each scenario is executed in a controlled way and documented using clear inputs, expected outcomes, and measurable observations. This makes it possible to assess how the system behaves before protection and how its security posture changes after defensive controls are introduced.

A major part of the proposed solution is the implementation of layered defensive mechanisms. These mechanisms are selected based on the vulnerabilities identified during testing and are intended to reduce the likelihood of unauthorized access and data exposure. The solution includes stronger authentication practices, encrypted communication through HTTPS, and brute-force protection mechanisms such as login attempt limiting and account lockout. Additionally, the system features custom firewall rules to control network access and an automated alerting system that flags suspicious activities. By applying these measures in stages, the system can be evaluated under different security conditions, such as unprotected operation, partially protected operation, and protected operation after full security enhancement.

Another important contribution of this solution is that it connects practical experimentation with measurable security improvement. Instead of discussing vulnerabilities only at a theoretical level, the proposed approach allows the project to observe actual attack behavior, analyze its effect on the system, and then verify whether the selected countermeasures reduce the success of those attacks. This creates a stronger basis for evaluating the effectiveness of the proposed defenses and provides a practical framework that can also be adapted to similar IoT camera environments.

Overall, the proposed solution provides a clear and systematic approach for assessing and improving the security of a Raspberry Pi camera system. It supports the research goals by combining controlled attack simulation, traffic analysis, and layered protection mechanisms within one integrated framework. In addition, it offers a practical model that can help researchers, students, and security practitioners better understand how camera systems behave under attack and how suitable security controls can be used to strengthen their resilience in real-world deployments.

5.2 Network Protocols

The given project is based on the analysis and testing of network protocols that have a direct impact on the security posture of smart cameras. The smart cameras generally depend on more than one communication layer such as video streaming protocols, control interfaces and firmware update channels, and all these are an attack surface.

The initial group of protocols applicable in this study is IP-based communication protocols, which are TCP and UDP. TCP features a reliable connection and is frequently utilized in a web-based control interface, portal of logins, and a firmware update system. UDP is more often, however, applied to video streaming because of its low latency, but it lacks reliability and encryption and is vulnerable to packet injection, spoofing, and replay attacks.

The second type is application-level protocols, specifically, HTTP, HTTPS, RTSP, ONVIF, and vendor-specific services. The HTTP also commonly uses administrative configuration but is not secure unless it is used with TLS. In HTTPS, there is the concept of encryption, but it can still be vulnerable in case the process of certificates validation is poorly executed. Real-time video streaming is done using RTSP, which in most cases is not encrypted and thus is a victim of eavesdropping and unauthorized access to feeds. On the same note, ONVIF, which is an interoperability standard applied to police the operations of cameras, can turn into an attack point when the credentials are weak or the configurations are not properly configured.

Other protocols that are involved in reconnaissance are UPnP, SSDP, and DHCP which help attackers to find the addresses and open services of devices. Misuse of protocols or improper configuration may leave internal services being exposed to outside networks.

These protocols were incorporated in this research when necessary in each attack or defense scenario only. As an example, RTSP has shown its analysis in experiments of video interception, and HTTP/HTTPS has been tested in authentication evaluation. The protocol analysis is directly connected with the knowledge of the attack vectors, assessment of exposure, and the findings of the effect of the layered security measures.

5.3 Methodology

In this project, the methodology used is in an organized, repetitive process which incorporates reconnaissance, exploitation, deployment of defense and re-assessment. Its aim is to come up with a repeatable architecture, which well represents the behavior of a smart camera when it is attacked and how its resilience is enhanced when defensive measures are applied. The methodology is based on the real-world penetration testing practices while maintaining academic rigor within a controlled experimental environment. It starts with environment preparation, where the smart camera is connected to a dedicated test network isolated from external and real-world systems. The testing machine is

configured with security tools such as Nmap, Wireshark, and custom Python scripts. This phase establishes a stable baseline environment and prepares the system for attack simulation. The second step, reconnaissance and vulnerability identification, involves scanning the camera to identify open ports, services, protocols, and possible misconfigurations. The results of this step help define the attack surface and determine suitable attack scenarios.The third step, controlled attack execution, involves testing predefined attack scenarios such as brute-force authentication attempts, RTSP access testing, network sniffing, and MITM interception. The fourth phase involves implementing defense mechanisms to evaluate the impact of each security layer on system resilience. These mechanisms include encryption, firewall rules, IDS, and strengthened authentication processes. Lastly, the evaluation stage involves re-executing the attacks after applying the security mechanisms to evaluate the effectiveness of the implemented defenses.

The following flowchart illustrates the overall methodology of the proposed system.

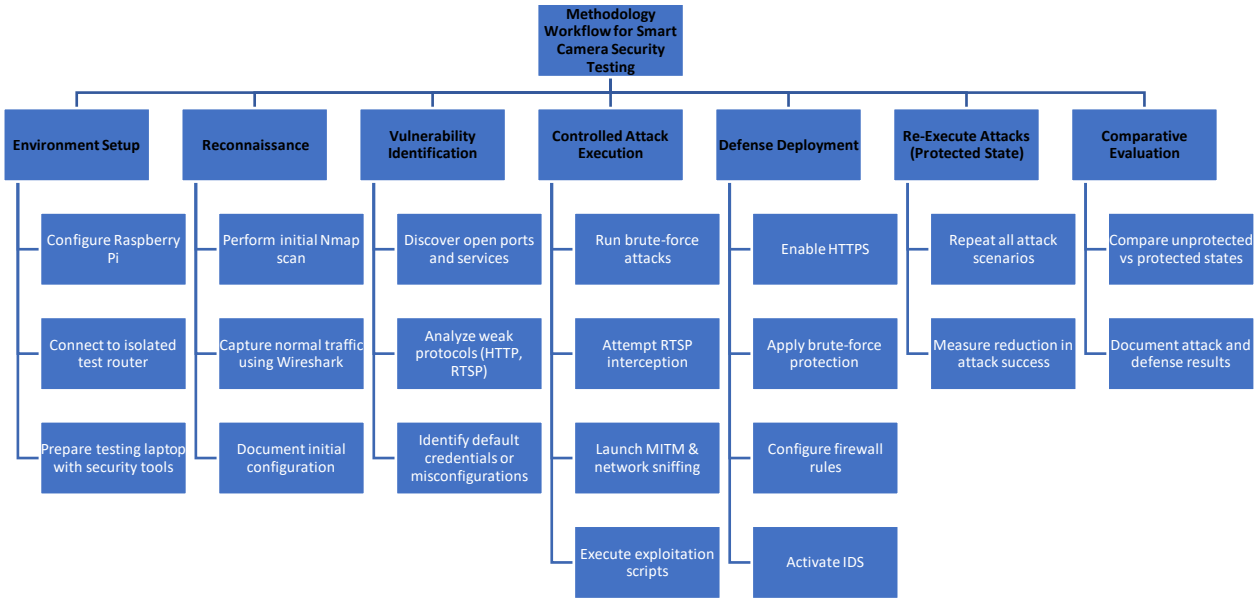


Figure 9 Methodology flow chart.

5.4 Procedures

This section presents the practical steps followed during the design of the system and the execution of the experiments. The procedure was organized to ensure clarity, repeatability, and safe testing within an isolated environment.

- Step 1: Environment Setup
  - The Raspberry Pi camera system was installed and configured as the primary target device. The required software components were prepared, including the streaming service and the web-based management interface. A Kali Linux virtual machine was used as the attacker environment to perform penetration testing activities within the test environment. All devices were connected to a dedicated test router within an isolated network.
- Step 2: Reconnaissance
  - A network scan was performed using Nmap to identify open ports, running services, and exposed communication channels. This phase helped define the attack surface and determine the most relevant services for further testing.

- Step 3: Vulnerability Identification

  Open ports, active services, weak communication protocols such as HTTP and RTSP, as well as default credentials and network misconfigurations, were identified.

- Step 4: Authentication Testing
  - Brute-force testing was performed against the web login interface and the RTSP authentication service in order to evaluate the strength of the configured credentials and identify weak authentication points.

- Step 5: MITM Attack Simulation
  - A controlled Man-in-the-Middle (MITM) scenario was performed using Bettercap to intercept traffic. This step was used to evaluate the risk of credential exposure and communication interception in the absence of proper encryption.
- Step 6: Network Traffic Analysis

- Wireshark was used to capture network traffic generated during system operation and attack execution. This phase focused on identifying exposed credentials, and unencrypted communication.

- Step 7: Defense Deployment
  - After identifying the main vulnerabilities, security mechanisms were applied to strengthen the system. These included HTTPS encryption for protecting the web interface, stronger authentication controls, brute-force protection mechanisms, and custom firewall rules to restrict unauthorized access.
- Step 8: Re-evaluation
  - The same attack scenarios were repeated after applying the security controls. The results were then compared to evaluate how the implemented protections affected attack success, credential exposure, and the overall security posture of the system.

5.5 User Interface Design

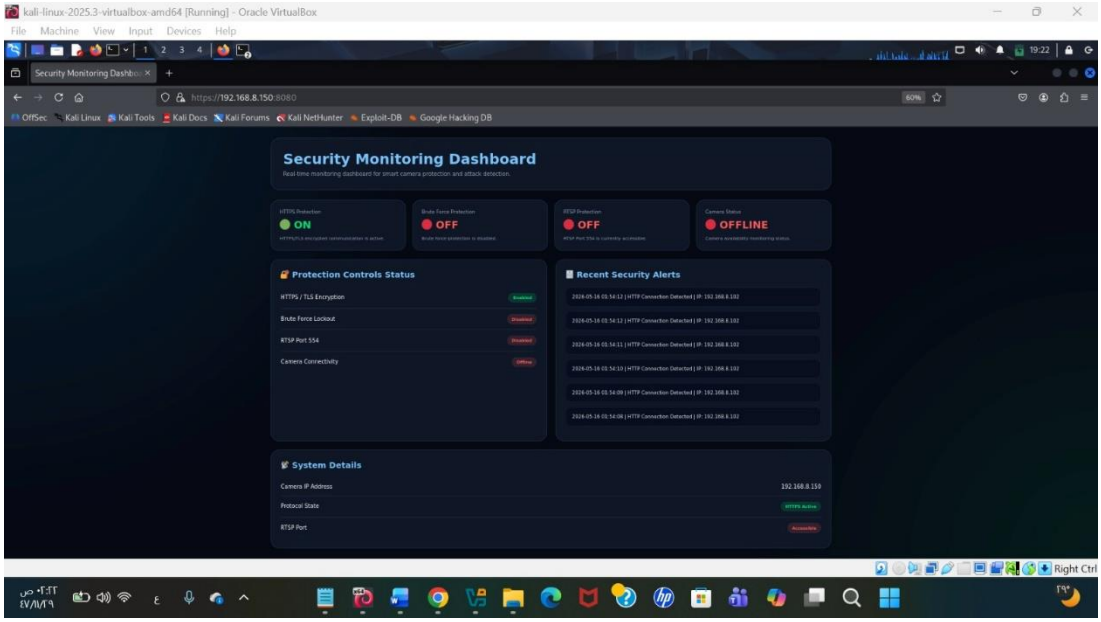


Figure 10 Homepage User Interface showing the main project functions.

The dashboard provides an interface for monitoring the security status of the Raspberry Pi camera system. It displays protection status, system details, and recent security alerts within the isolated test environment.

5.6 Summary

This chapter presented the proposed solution for securing the Raspberry Pi camera system through a structured, controlled, and repeatable testing approach. It explained how the proposed solution was designed to evaluate system vulnerabilities, simulate realistic attack scenarios, and apply suitable defensive mechanisms in order to improve the overall security posture of the system. The chapter described the main methodology followed in the project, including environment setup, reconnaissance, controlled attack simulation, traffic analysis, defense implementation, and security evaluation. It also clarified how the Raspberry Pi camera system was used as the main target device within an isolated laboratory environment, allowing the project to perform safe and ethical testing without affecting external systems. In addition, the proposed solution focused on practical attack scenarios that directly relate to the implemented project work, such as brute-force attempts against authentication interfaces, RTSP access testing, network sniffing, and Man-in-the-Middle (MITM) interception. These attack scenarios helped demonstrate how weak authentication and unencrypted communication can expose sensitive data such as login credentials and streaming-related traffic. Furthermore, the chapter explained the role of layered protection mechanisms in reducing these risks. These mechanisms included stronger authentication practices, HTTPS encryption to secure communication, brute-force protection mechanisms, and custom firewall rules to restrict unauthorized access. By comparing system behavior before and after applying these controls, the project was able to evaluate the effectiveness of the selected security measures. Overall, this chapter provided a clear view of how the proposed solution integrates attack simulation, traffic analysis, and defensive controls into one practical framework.

Chapter 6. Implementation

Implementation

This part of the report explains how we actually set up our project environment and prepared the hardware and software for the testing phase.

6.1 Lab Setup & Environment

We designed the lab to act like a real-world smart camera system. We made sure it was a safe and isolated environment to prevent any impact on outside networks, ensuring that our methodology follows ethical hacking standards by avoiding any unauthorized access to external systems.

6.1.1 Target Device Configuration (Raspberry Pi)

We used a Raspberry Pi 4 (8GB RAM) as our target device to represent a smart security camera. To capture the live feed, we equipped it with a 5MP Raspberry Pi Camera Module. Here is how we set it up:



Figure 11 Lab hardware setup including the Raspberry Pi, camera module, and the dedicated router.

- **OS Installation:** We installed the Raspberry Pi OS on an SD card and finished the basic setup.

- **System Control & Remote Access:** We utilized the Terminal to establish a secure connection via SSH (Secure Shell). This allowed us to manage the device remotely and initialize the necessary services, such as MediaMTX for RTSP streaming and the Nginx/PHP environment for the web dashboard.

- **Streaming & RTSP Setup (MediaMTX):**

  - **Streaming Server:** We utilized MediaMTX as our primary media server to handle the camera's live stream. We chose this server because it is a simple and efficient solution that supports multiple protocols including RTSP and WebRTC. It allowed us to easily bridge the physical camera module with our web-based dashboard.

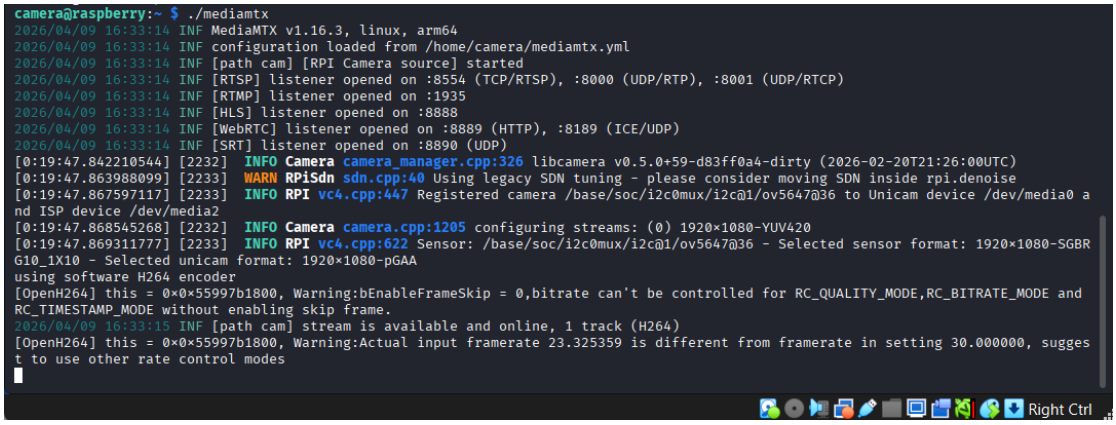


Figure 12 MediaMTX terminal output confirming the server has loaded the configuration, started the camera stream, and opened the RTSP listener on port 554.

Brute-Force Testing Setup: Within the MediaMTX configuration, we enabled the authentication feature. We manually set a username and password for the RTSP stream to

create a controlled vulnerability. This was done to test if security tools like Hydra could successfully perform a brute-force attack to gain unauthorized access.

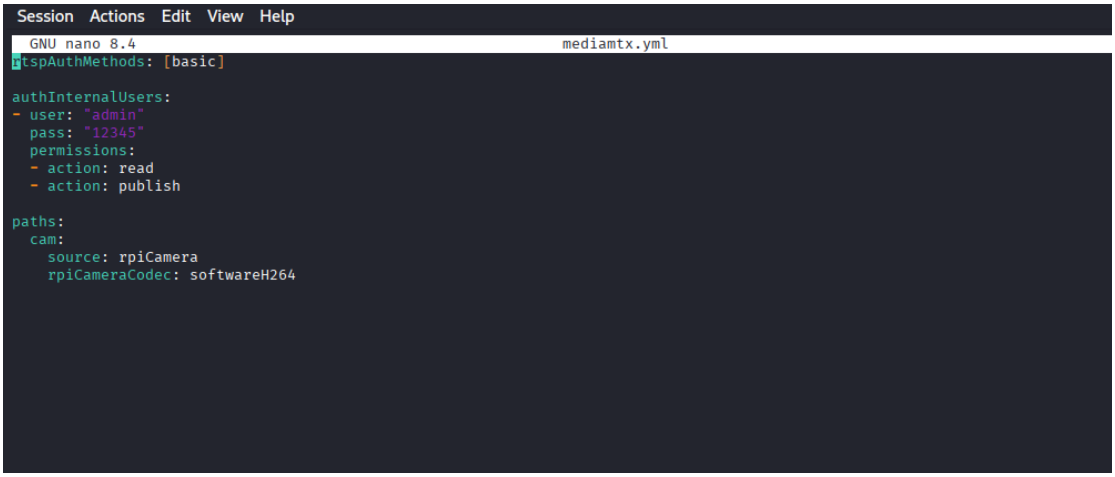


Figure 13 MediaMTX configuration file (mediamtx.yml) showing the manual setup of basic authentication (username: admin, password: 12345) to require credentials for RTSP stream access.

6.2 Web Dashboard Development:

Technology Stack:

- Frontend: HTML, CSS
- Backend: PHP 8.4
- Web Server: Nginx
- Database: SQLite3

Authentication Interface:

We designed a login page (Figure 15) as the entry point for the camera system.

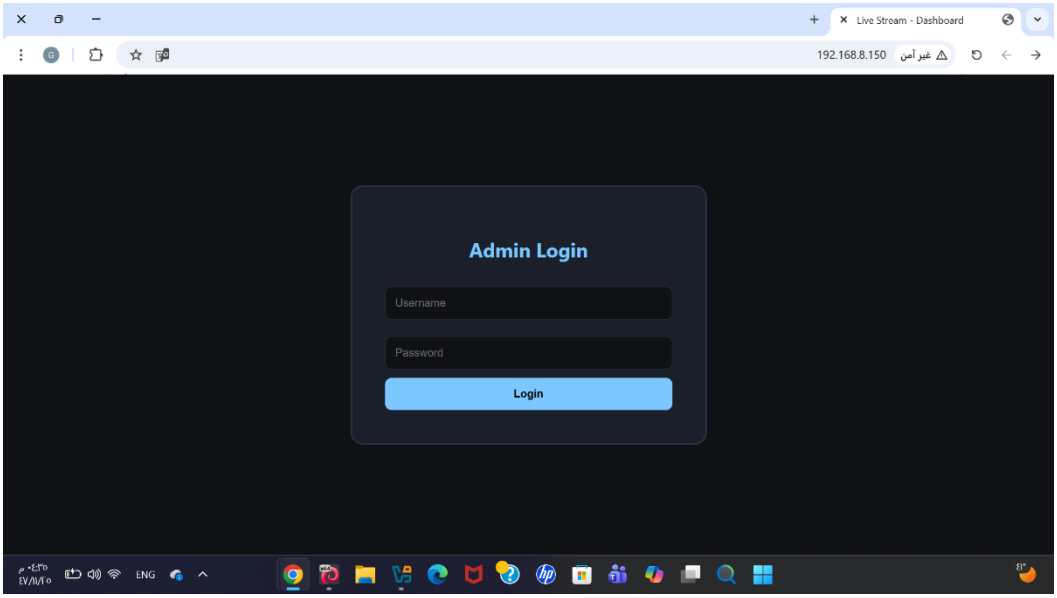


Figure 14 The web management authentication interface developed for the Smart Camera Dashboard.

Backend Logic:

The server-side functionality is hosted on an Nginx web server and built using PHP 8.4 to manage administrative operations and user access. It features a session-based system for the camera dashboard, utilizing an SQLite3 relational

database for storing credentials. This architecture supports real-time data interactions, allowing the administrator to update access details through a dedicated settings interface, with all changes committed directly to the `system_users.db` file via the Nginx-PHP integration.

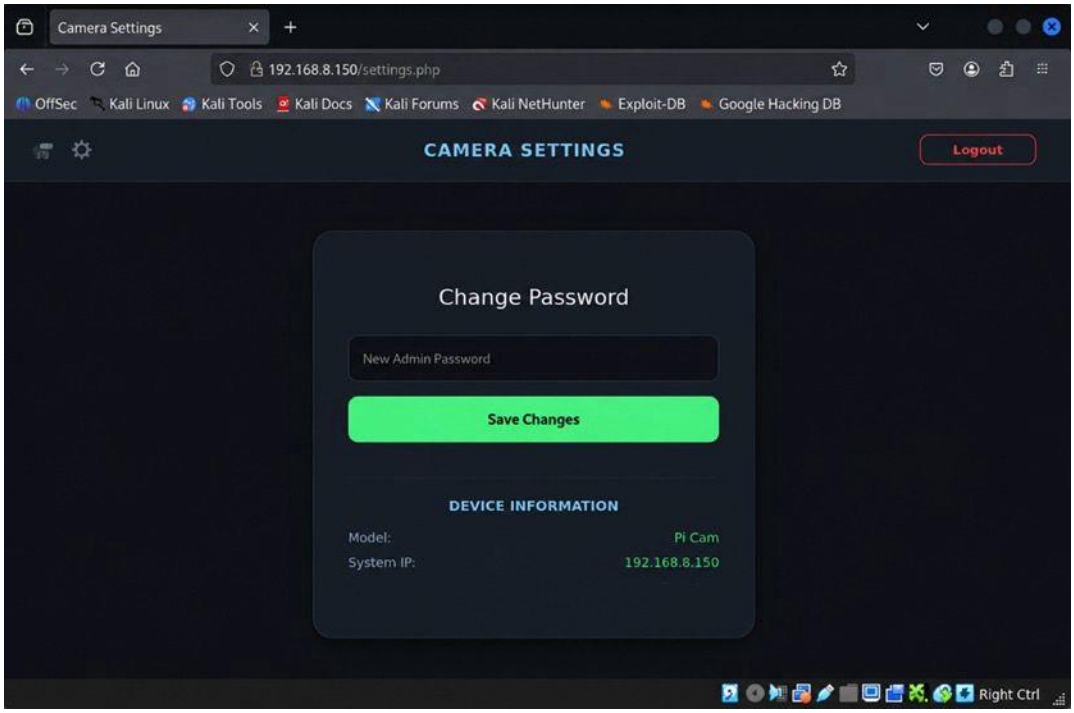


Figure 15 The System Settings interface of the Smart Camera Dashboard, featuring password management and device information.

Frontend & UI: We designed a custom dashboard using HTML and CSS. The interface was styled to look like a professional camera monitoring system.

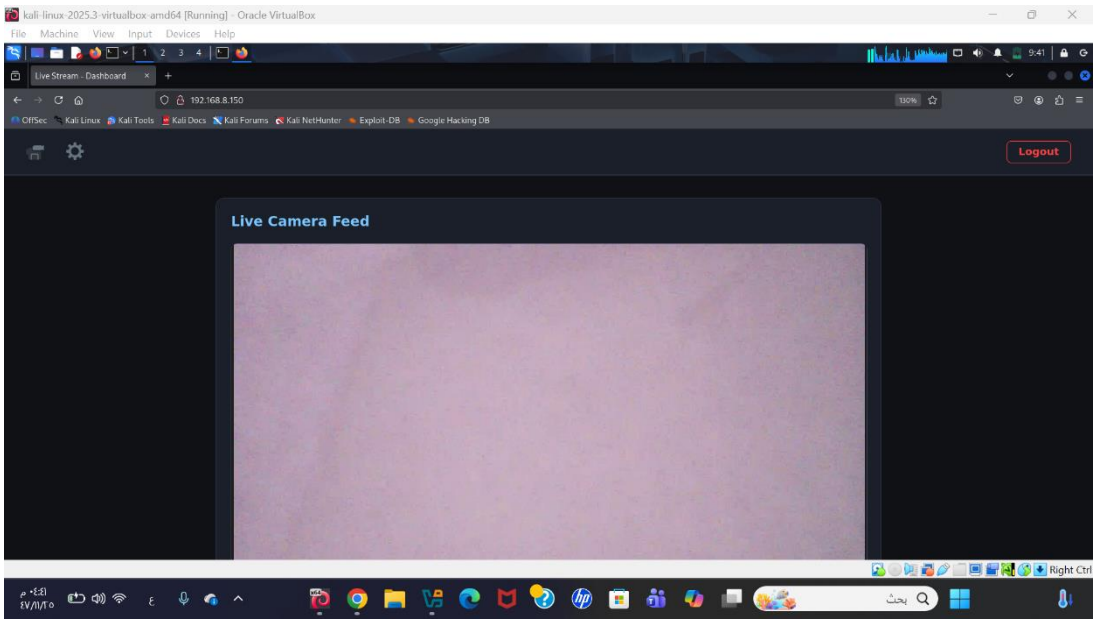


Figure 16 The frontend user interface showing the live embedded camera feed within the management dashboard.

Live Feed Integration: The RTSP stream was received and processed by MediaMTX, then exposed as an HTTP-accessible endpoint, which was embedded into the web dashboard using an iframe.

```
<div class="container">
    <div class="card">
        <h3 style="margin-top:0; color:#7cc7ff;">Live Camera Feed</h3>
        <div class="video-feed">
            <iframe src="http://admin:12345@192.168.8.150:8888/cam" allowfullscreen></iframe>
        </div>
    </div>
</div>
```

Figure 17 Code snippet showing the iframe integration for the live camera stream.

6.3 Attacker Platform (Kali Linux)

For the attack side, we used Kali Linux because it has all the security tools we need.

- **Virtual Machine:** Kali Linux was used within VirtualBox to isolate the attack environment from the host system, ensuring safe and controlled testing.

- **Tools Used:** We utilized tools such as Nmap (for network scanning), Hydra (for password brute-force attacks), Wireshark (for capturing and analyzing network traffic), and Bettercap (for performing Man-in-the-Middle attacks and intercepting network traffic).

6.4 Network Isolation and Secure Testing Environment

To ensure that all testing activities remained within an ethical and controlled environment, a dedicated and isolated network infrastructure was implemented:

- **Physical Segmentation:** A dedicated router was used to connect all project devices within a separate network. The Raspberry Pi was connected via an Ethernet cable to ensure a stable connection, while Kali Linux was used on two separate virtual machines within the same isolated network.

- **Virtual Isolation:** Kali Linux was used within VirtualBox to isolate the testing environments from the host system, ensuring safe and controlled testing. One Kali

machine was used to perform attacks, while the other was used to remotely access and manage the Raspberry Pi via SSH.

6.5 Reconnaissance

This phase focused on identifying accessible services within the target environment to determine potential entry points. Network scanning was performed using Nmap on the target device (192.168.8.150). The results revealed multiple open ports and active services, including SSH (22), HTTP (80), RTSP (554), and RTMP (1935).

These findings indicated several exposed services, particularly HTTP and RTSP, which were later selected as primary targets for further attack scenarios.

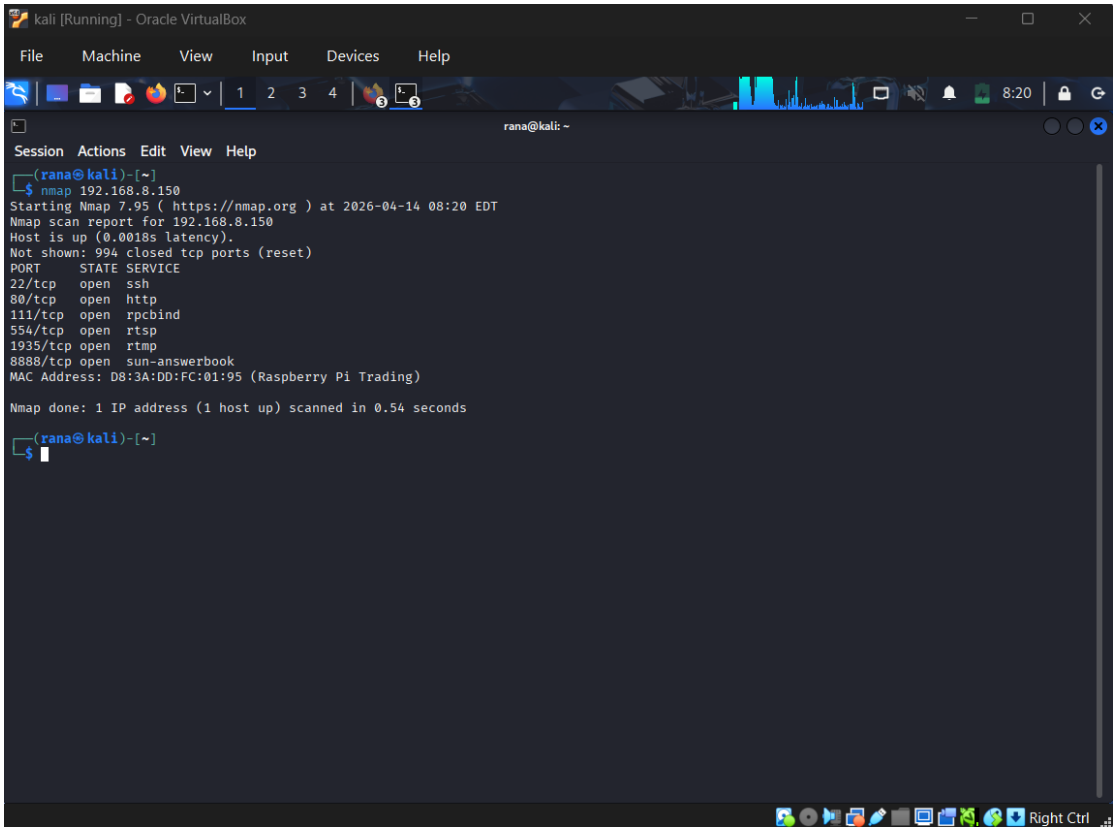


Figure 18 Nmap scan results showing open services and exposed ports on the Raspberry Pi smart camera system.

Following the identification of open HTTP services, the web interface was accessed via a browser. A login page was observed, indicating a user authentication point.

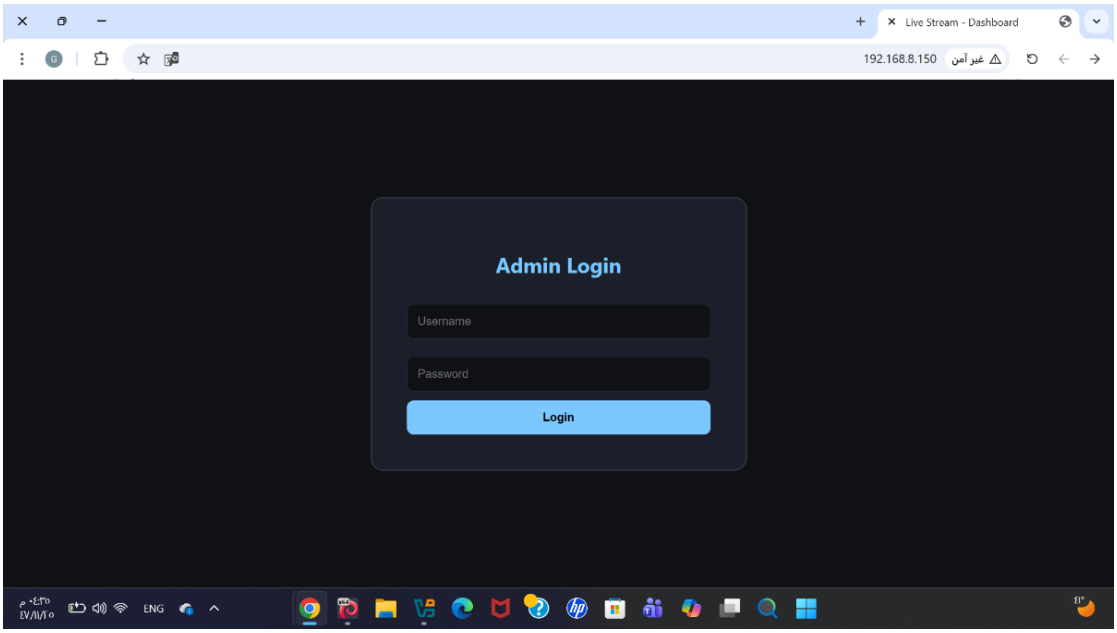


Figure 19 The web interface was accessed via a browser, where a login page was observed.

The RTSP service was also tested using a VideoLAN Client (VLC) media player. An attempt to access the stream without credentials was unsuccessful.

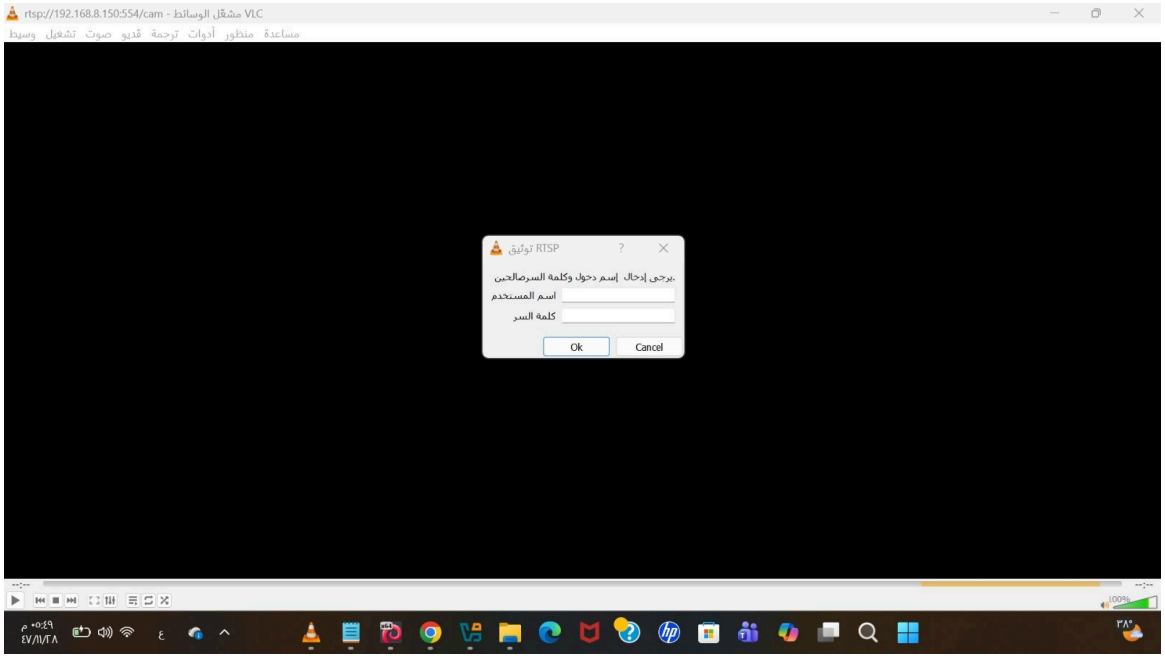


Figure 20 Attempt to access the RTSP stream using VLC without credentials.

6.6 Attacks Phase

Following the reconnaissance phase, several attack scenarios were conducted to evaluate the security of the system. The identified services were targeted to assess potential vulnerabilities and demonstrate possible exploitation methods.

6.6.1 Brute Force Attack

A brute force attack was performed on the web login interface. The system response “Invalid credentials” was used to recognize failed login attempts.Common username and password lists were prepared and used with Hydra to try multiple login attempts automatically. The attack was able to find valid credentials, showing that the system is vulnerable to brute force attacks.

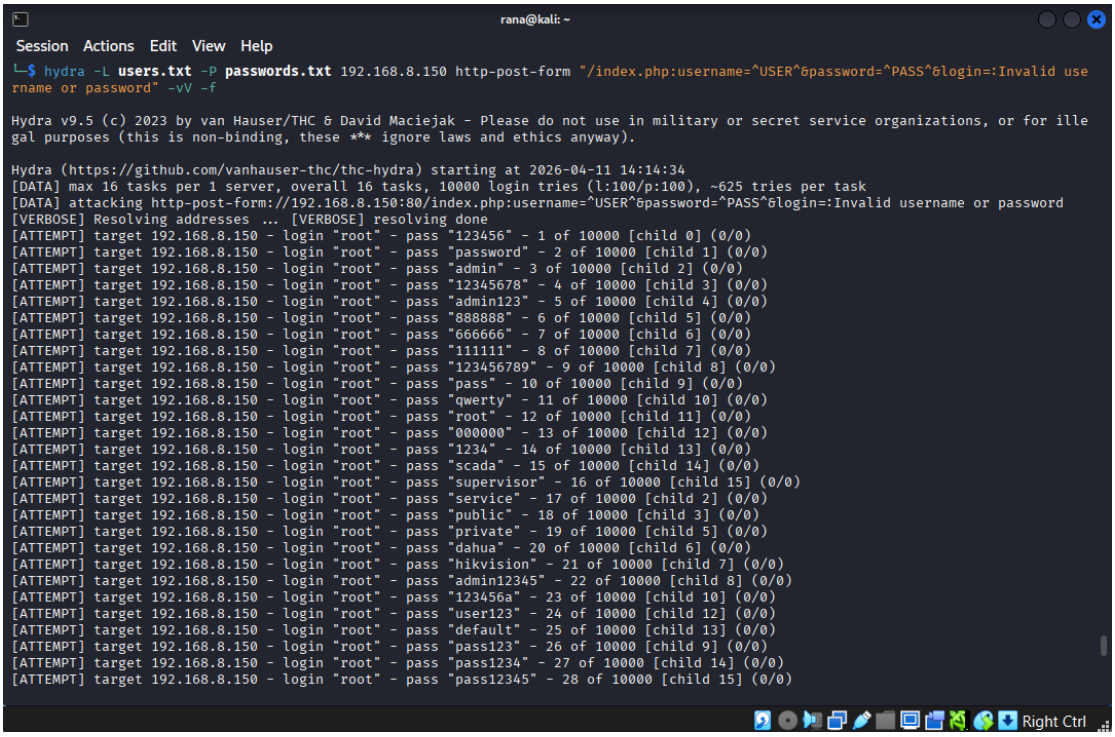


Figure 21 Hydra tool executing a brute force attack on the login interface.

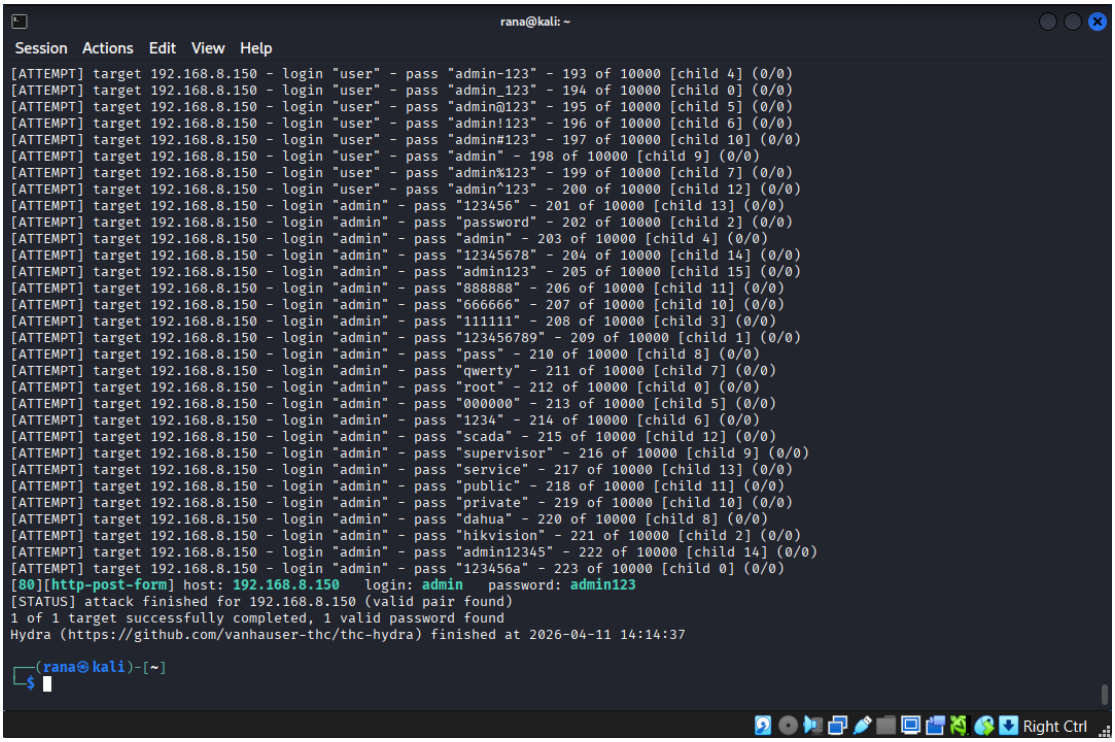


Figure 22 Successful brute force result revealing valid credentials.

6.6.2 RTSP Brute Force Attack

A brute force attack was performed on the RTSP service using a Python script. The script was used to try multiple username and password combinations to access the stream.The attack was successful and revealed valid credentials, showing that the RTSP service is vulnerable to brute force attacks.

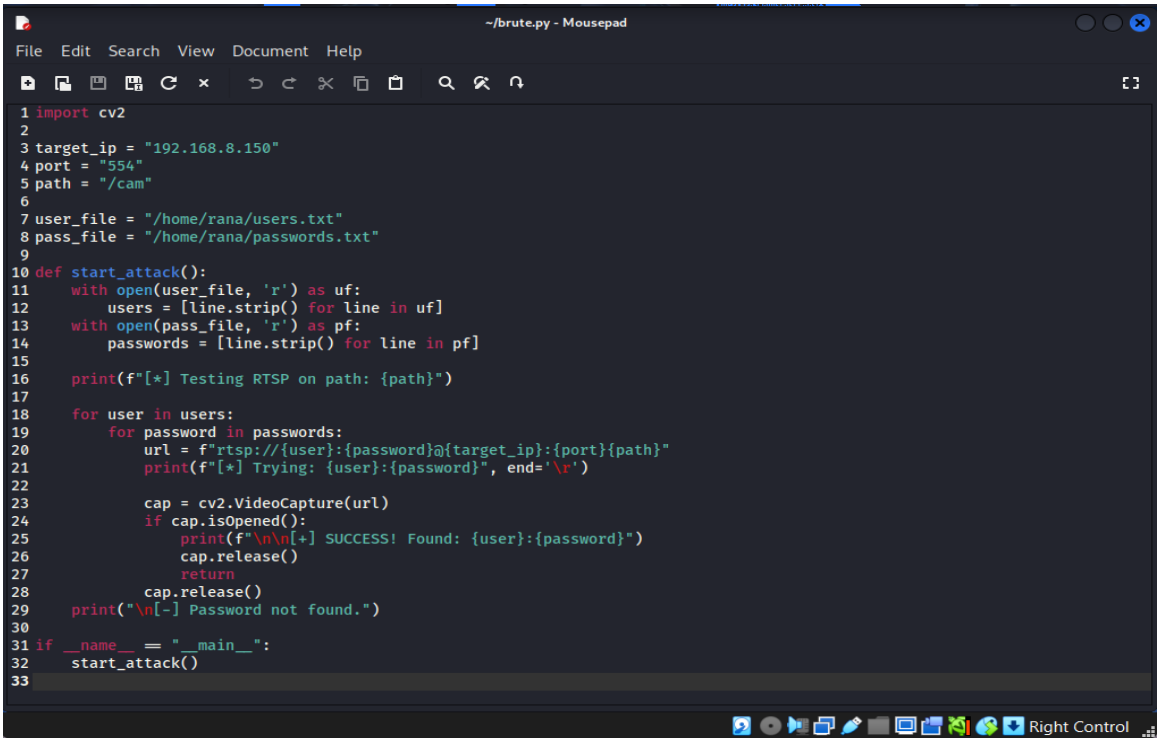


Figure 23 Python script used to perform the RTSP brute force attack.

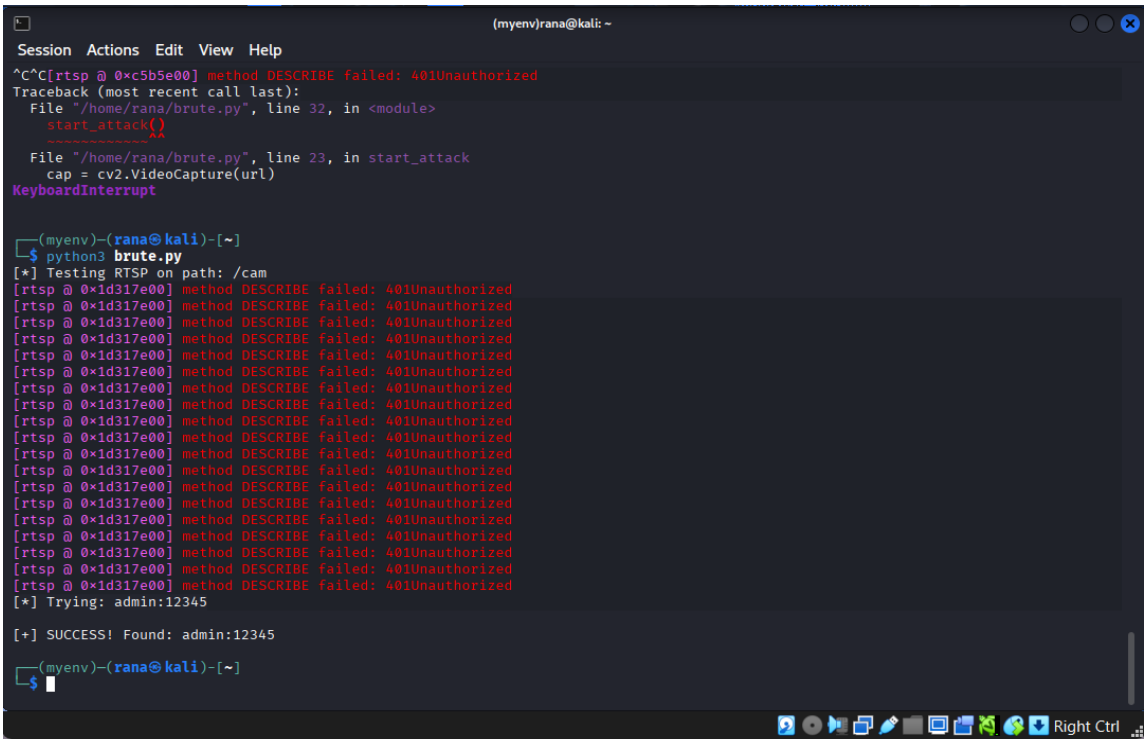


Figure 24 Successful RTSP brute force result revealing valid credentials.

6.6.3 Network Sniffing Attack

A network sniffing attack was performed using Wireshark to monitor the traffic between the client and the server. HTTP packets were analyzed, focusing on POST requests. During the analysis, sensitive data was observed in plain text, including the username and password transmitted through the login request. In addition, parts of the video stream were also visible within the captured traffic. This demonstrates that the lack of encryption allows attackers to intercept sensitive information directly from network traffic.

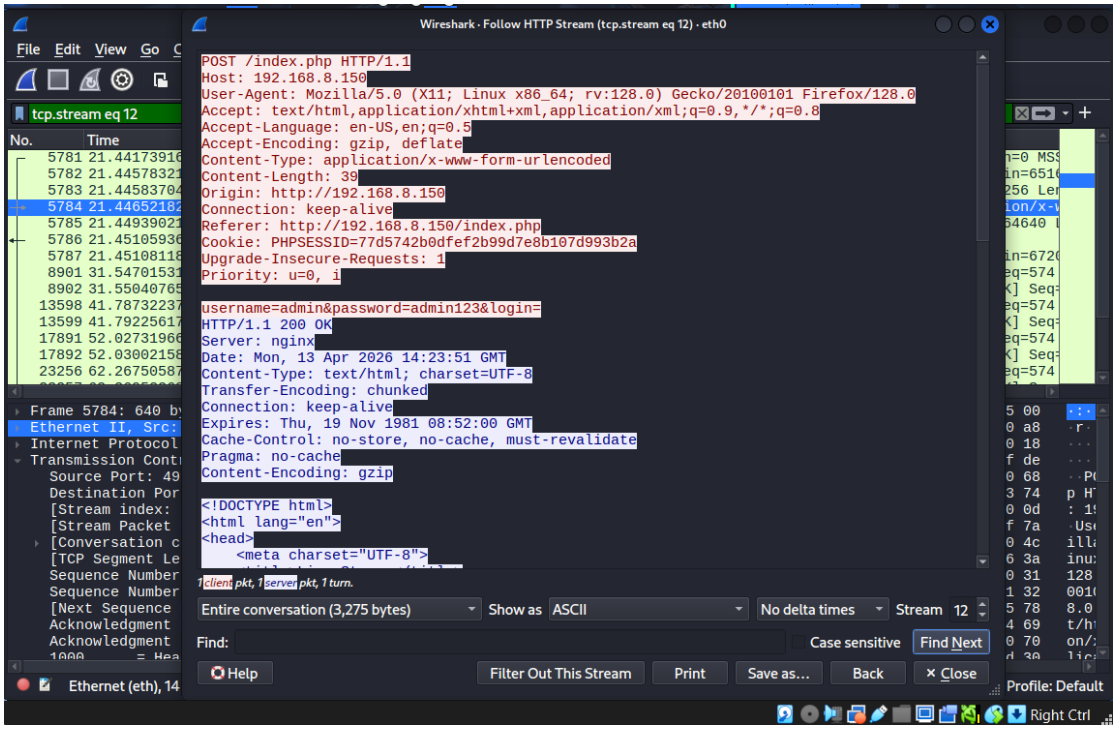


Figure 25 Captured HTTP POST request showing username and password in plain text.



Figure 26 Captured network traffic showing video data transmitted over HTTP.

6.6.4 Man-in-the-Middle (MITM) Attack

A Man-in-the-Middle attack was performed using Bettercap as a network interception tool to capture traffic between the client and the server. ARP spoofing was used to position the attacker between them, allowing traffic monitoring. During the attack, when the user attempted to log in to the web interface, the HTTP POST request was captured. The transmitted credentials, including the username and password, were visible in plain text. In addition, requests related to the video stream were also observed, showing that the streaming service uses the same communication channel. This demonstrates that the lack of encryption allows attackers to directly intercept sensitive information, including both login credentials and streaming access data, without the need for brute force attacks.

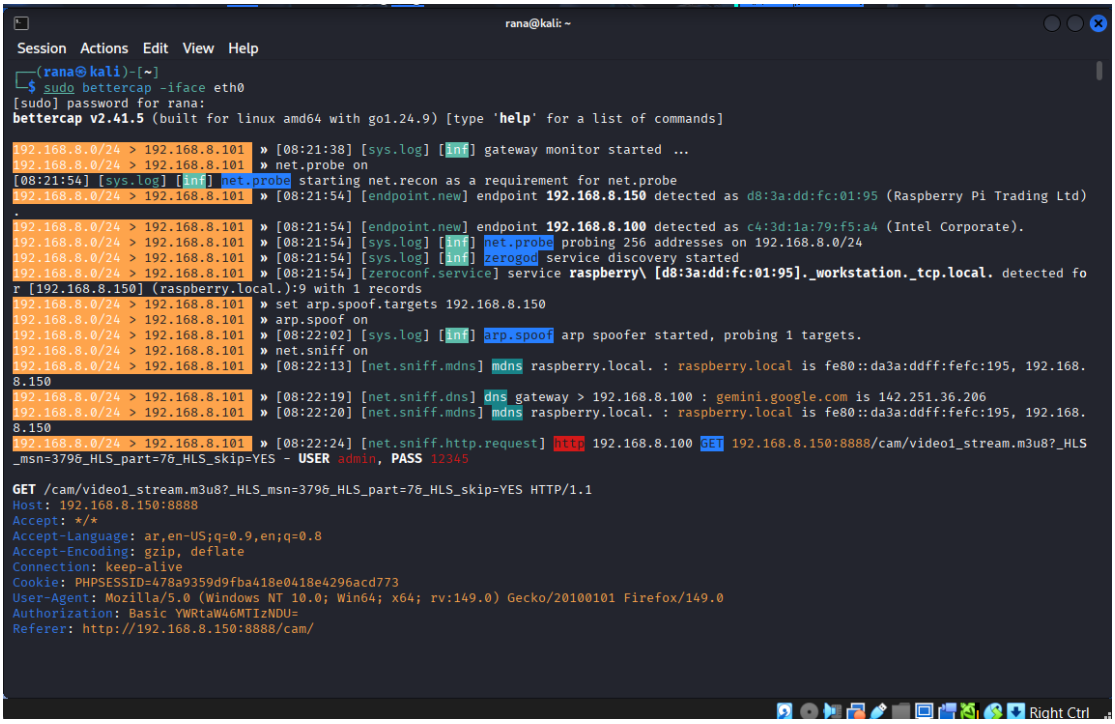


Figure 27 Bettercap performing ARP spoofing and traffic interception.

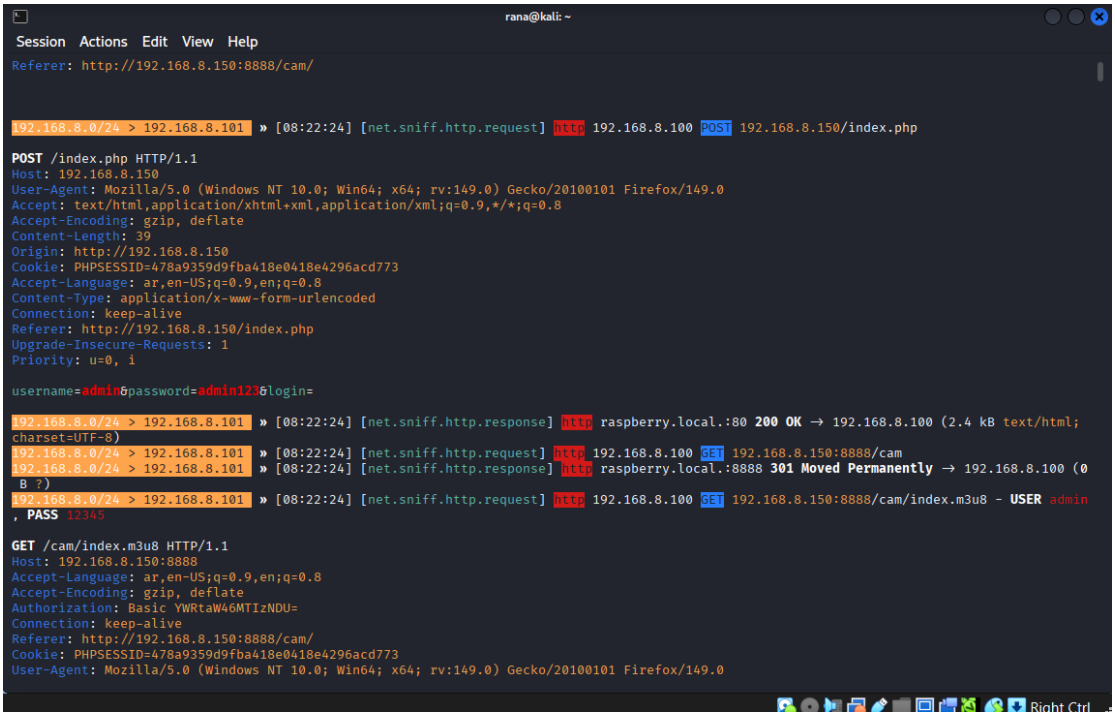


Figure 28 Captured HTTP request during the MITM attack.

6.7 Protection Phase

After conducting multiple attack scenarios, including brute-force, Man-in-the-Middle (MITM), Network Sniffing, and RTSP attacks, several defensive mechanisms were implemented to improve the security of the smart camera system. The protection phase focused on authentication hardening, secure communication, attack prevention, and real-time attack detection.

6.7.1 Brute Force Attack Protection

During the attack phase, it was observed that the login system was vulnerable to brute-force attacks using tools such as Hydra. Attackers were able to repeatedly attempt different username and password combinations in order to gain unauthorized access.

Implementation of Protection

To mitigate this vulnerability, a login protection mechanism was implemented using PHP sessions, temporary account lockout, Multi-Factor Authentication (MFA), and real-time email alert notifications.

Attempt Counter Initialization

The system initializes failed login attempt counters using PHP sessions:

```php
if (!isset($_SESSION['attempts'])) {
$_SESSION['attempts'] = 0;
}

if (!isset($_SESSION['lockout_time'])) {
$_SESSION['lockout_time'] = 0;
}
```

Failed Login Attempt Tracking

Each failed login attempt increases the counter when brute-force protection is enabled:

```php
            if ($global_protection) {
                $_SESSION['attempts']++;
```

Account Lockout Protection

If the number of failed login attempts reaches 5 attempts or more, the system temporarily blocks further login attempts for 30 seconds:

```
        if ($_SESSION['attempts'] >= 5) {
            $_SESSION['lockout_time'] = time() + 30;
        }
```

Lock Duration Control

The system checks whether the lockout period is still active before allowing additional login attempts:

```
$current_time = time();
if ($global_protection && $current_time < $_SESSION['lockout_time']) {
    $remaining = $_SESSION['lockout_time'] - $current_time;
    $msg = "⛔ Too many failed attempts. Try again after $remaining seconds.";
}
```

This mechanism helps mitigate brute-force attacks by limiting repeated authentication attempts.

Multi-Factor Authentication (MFA)

To further strengthen authentication security, the system implements Multi-Factor Authentication (MFA). After successful username and password verification, a 6-digit verification code is generated and sent to the registered email address.

```
        if ($user['mfa_enabled'] == 1) {
            $_SESSION['mfa_user'] = $user['username'];
            $_SESSION['mfa_code'] = rand(100000, 999999);
            $_SESSION['mfa_step'] = true;
            $to = $user['email'];
            mail($to, "Security Verification Code", "Your code is: " . $_SESSION['mfa_code']);
        }
```

The user must enter the verification code to complete the login process successfully.

Email Alert Notification

When multiple failed login attempts are detected, the system automatically sends a real-time email alert to the administrator:

```
    $to = "woun2026@gmail.com";
    $subject = "🚨 Brute Force Attack Detected";

    $message = "
Warning!

Multiple failed login attempts detected.

IP Address: " . $_SERVER['REMOTE_ADDR'] . "
Attempts: " . $_SESSION['attempts'] . "
Time: " . date("Y-m-d H:i:s") . "

The system temporarily locked login access.
";

$headers = "From: Raspberry Camera <security@picam.local>";

    mail($to, $subject, $message, $headers);
}
```

The email alert includes:

- Attacker IP address
- Number of failed attempts
- Detection timestamp

Verification Result

After applying the protection mechanisms:

- Brute-force attacks were blocked after limited login attempts.
- The system temporarily locked authentication access for 30 seconds.
- Attackers could no longer repeatedly guess passwords freely.
- MFA added an additional authentication security layer.
- The administrator received real-time email alerts when suspicious activity was detected.

These results confirm that the implemented protection mechanisms significantly improved authentication security against brute-force attacks.

6.7.2 Man-in-the-Middle (MITM) & Network Sniffing Protection

During the attack phase, the system was tested against Man-in-the-Middle (MITM) and Network Sniffing attacks. Since HTTP communication is not encrypted, attackers may intercept and monitor network traffic transmitted between the user and the smart camera

system. Using network sniffing techniques, it was possible to observe unencrypted communication traffic when the platform was accessed through HTTP. Sensitive information and transmitted requests could potentially be intercepted by attackers connected to the same network.

Implementation of Protection

To mitigate this vulnerability, HTTPS protection and HTTP access detection mechanisms were implemented to secure communication and detect insecure access attempts.

HTTPS Protection Activation

The following script was implemented to enable HTTPS protection dynamically:

```
#!/bin/bash
cat > /etc/nginx/sites-available/default <<'EOF'
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
return 302 https://$host$request_uri;
}

server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;

    root /var/www/html;
    index index.php;
    server_name _;

    ssl_certificate /etc/ssl/certs/cert.pem;
    ssl_certificate_key /etc/ssl/private/key.pem;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.4-fpm.sock;
    }
}
EOF

nginx -t && systemctl reload nginx

echo "HTTPS Enabled"
```

The HTTPS protection mechanism was implemented using a custom shell script that automatically configures the Nginx web server, enables SSL/TLS encryption, and redirects all HTTP requests to HTTPS communication. The script also loads SSL certificates and activates secure communication on Port 443.

HTTPS Protection Deactivation

The following script was implemented to disable HTTPS protection for testing purposes:

```
#!/bin/bash
cat > /etc/nginx/sites-available/default <<'EOF'
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    root /var/www/html;
    index index.php;
    server_name _;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.4-fpm.sock;
    }
}
EOF

nginx -t && systemctl reload nginx
echo "HTTPS Disabled"
```

The disable HTTPS script restores the Nginx configuration to HTTP mode by removing SSL/TLS enforcement and reloading the Nginx service. This script was used during testing to compare system behavior before and after enabling HTTPS protection.

HTTPS Status Detection

The system checks whether HTTPS is currently active:

```
$is_https = (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off');
```

This value is used by the dashboard to display the current HTTPS protection status

HTTP Connection Detection

The system continuously checks whether users access the platform using HTTP instead of HTTPS:

```
if (empty($_SERVER['HTTPS']) || $_SERVER['HTTPS'] === 'off') {
```

If an insecure HTTP connection is detected, the system automatically sends a real-time email alert to the administrator.

Email Alert Notification

```
        $to = "woun2026@gmail.com";
        $subject = "⚠ HTTP Connection Detected";

        $message = "
Warning!

An insecure HTTP connection was detected.

IP Address: " . $_SERVER['REMOTE_ADDR'] . "
Time: " . date("Y-m-d H:i:s") . "

The user accessed the system without HTTPS encryption.
";

$headers = "From: Raspberry Camera <security@picam.local>";

        mail($to, $subject, $message, $headers);
```

The email alert includes:

- Source IP address
- Detection timestamp
- Warning about insecure communication

Verification Result

The protection mechanism was verified by accessing the platform through HTTP while monitoring network traffic using sniffing techniques.

Results:

- HTTP access attempts triggered automatic email alerts.
- HTTPS protection redirected users from HTTP to HTTPS.
- Sensitive data was no longer visible during network sniffing because communication became encrypted.
- HTTPS protection could be dynamically enabled or disabled through the dashboard interface.

These results confirm that the implemented HTTPS protection successfully mitigates MITM and network sniffing attacks.

6.7.3 RTSP Attack Protection

During the attack phase, the smart camera system was tested against RTSP-based attacks targeting Port 554. Since RTSP is commonly used for camera streaming services, attackers

may attempt to scan, access, or exploit the RTSP service to gain unauthorized access to video streams. Using network scanning tools such as Nmap, it was possible to identify and access the RTSP port when protection was disabled.

Implementation of Protection

To mitigate this vulnerability, firewall-based RTSP protection and real-time attack detection mechanisms were implemented using iptables and system log monitoring.

RTSP Protection Activation

The following script was implemented to enable RTSP protection dynamically:

```
#!/bin/bash

while iptables -C INPUT -p tcp --dport 554 -j ACCEPT 2>/dev/null; do
    iptables -D INPUT -p tcp --dport 554 -j ACCEPT
done

while iptables -C INPUT -p tcp --dport 554 -j DROP 2>/dev/null; do
    iptables -D INPUT -p tcp --dport 554 -j DROP
done

while iptables -C INPUT -p tcp --dport 554 -j LOG --log-prefix "RTSP_ATTACK: " 2>/dev/null; do
    iptables -D INPUT -p tcp --dport 554 -j LOG --log-prefix "RTSP_ATTACK: "
done

iptables -I INPUT 1 -p tcp --dport 554 -j DROP
iptables -I INPUT 1 -p tcp --dport 554 -j LOG --log-prefix "RTSP_ATTACK: "

echo "RTSP Protection Enabled"
```

This script:

- Blocks unauthorized access to Port 554
- Logs suspicious RTSP access attempts
- Enables firewall-based RTSP protection

RTSP Protection Deactivation

The following script was implemented to disable RTSP protection:

```
#!/bin/bash


while iptables -C INPUT -p tcp --dport 554 -j DROP 2>/dev/null; do
    iptables -D INPUT -p tcp --dport 554 -j DROP
done

while iptables -C INPUT -p tcp --dport 554 -j ACCEPT 2>/dev/null; do
    iptables -D INPUT -p tcp --dport 554 -j ACCEPT
done


iptables -I INPUT 1 -p tcp --dport 554 -j ACCEPT

echo "RTSP Protection Disabled"
```

This script removes firewall restrictions and restores RTSP accessibility for testing purposes.

RTSP Attack Detection

The system continuously monitors RTSP attack attempts through firewall logs and automatically generates alert notifications.

```
journalctl -f | while read line; do
    echo "$line" | grep "RTSP_ATTACK" >/dev/null
```

Email Alert Notification

When suspicious RTSP activity is detected, the system automatically sends a real-time email alert to the administrator.

```
 | mail -s "🚨 RTSP Attack Detected" "$EMAIL"
```

The email alert includes:

- Attacker IP address
- Detection timestamp
- RTSP attack warning message

Verification Result

The protection mechanism was verified using network scanning tools such as Nmap to test RTSP accessibility on Port 554.

Results:

- When RTSP protection was disabled, Port 554 appeared as open or accessible.
- When RTSP protection was enabled, Port 554 appeared as filtered due to firewall restrictions.
- RTSP scanning attempts triggered automatic email alerts.
- Unauthorized access to the RTSP service was successfully blocked.

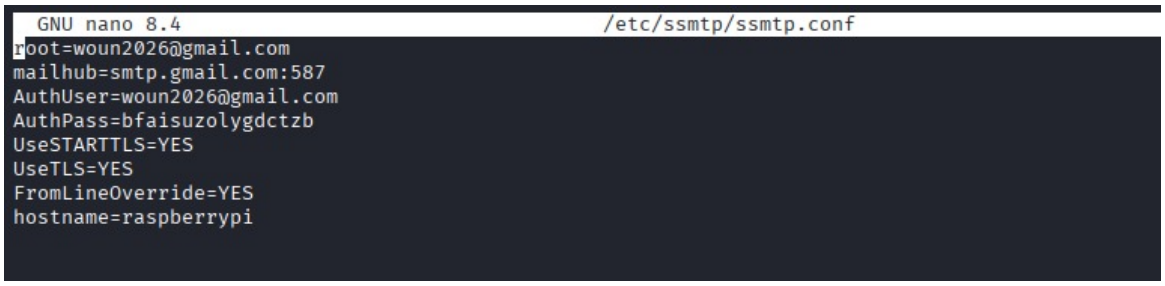
These results confirm that the implemented RTSP protection mechanism successfully mitigates unauthorized RTSP access and reconnaissance attacks targeting the smart camera system.

6.7.4 Email Notification Configuration

To support real-time security monitoring and Multi-Factor Authentication (MFA), the Raspberry Pi system was configured to send automated email notifications using the ssmtp mail service.

The SMTP configuration file was modified to allow the system to send emails through a Gmail account.

Mail Configuration



This configuration enabled the system to send:

- MFA verification codes
- Brute-force attack alerts
- HTTP insecure connection alerts
- RTSP attack detection alerts

The email notification mechanism improved real-time attack awareness and allowed the administrator to monitor suspicious activities immediately after detection.

6.8 Security Monitoring Dashboard

To improve system monitoring and security visibility, a security monitoring dashboard was developed for the smart camera environment. The dashboard provides an interface for monitoring protection status and detected security events during testing and evaluation.

The monitoring dashboard displays:

- HTTPS/TLS protection status
- RTSP protection status
- Brute-force protection status
- Camera connectivity status
- Recent security alerts and attack events
- System details and protocol status

The dashboard reflects the current security configuration of the system and records detected events with timestamps to support attack monitoring and security evaluation within the isolated test environment.

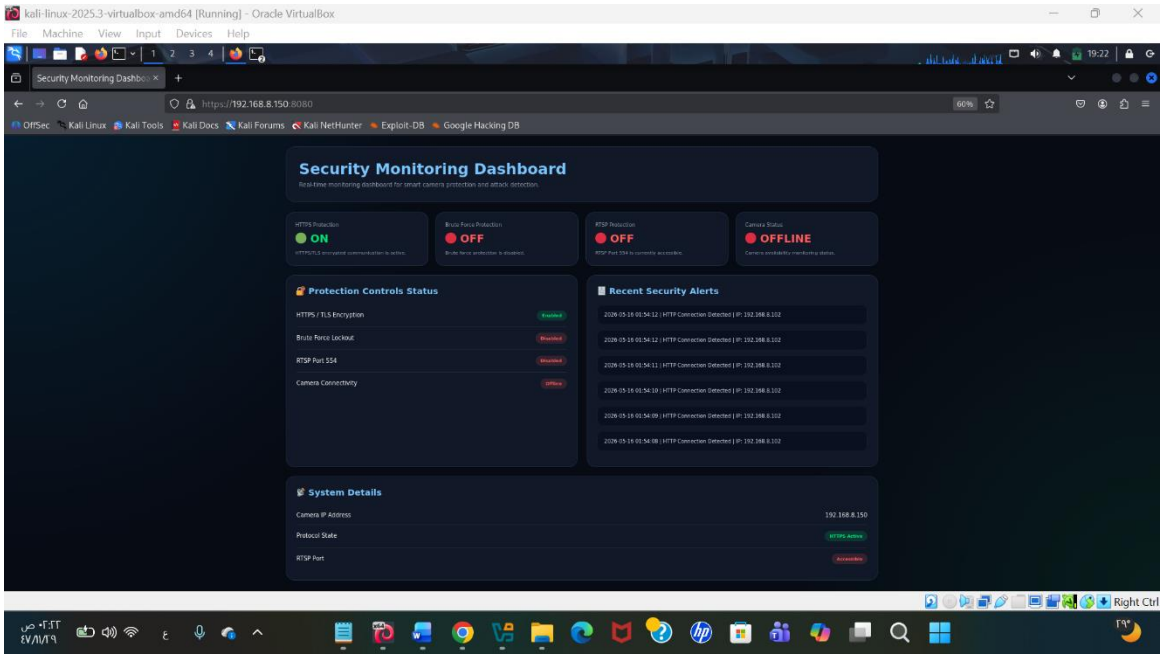


Figure 29 Security Monitoring Dashboard displaying protection status and recent security alerts.

The dashboard successfully displayed real-time security monitoring information including attack alerts, and protection states.

Chapter
7. Testing

7.1 Brute Force Attack Testing

To evaluate the effectiveness of the implemented brute-force protection mechanism, multiple authentication attack scenarios were performed using a custom Python brute-force script and repeated invalid login attempts. The objective of the testing phase was to compare the authentication system behavior before and after enabling the implemented security mechanisms.

7.1.1 Brute Force Attack Before Protection

Before enabling brute-force protection, the authentication system allowed unlimited login attempts. During the attack simulation, the attacker repeatedly attempted different username and password combinations until valid credentials were discovered successfully. This demonstrated that the login system was vulnerable to automated password guessing attacks.

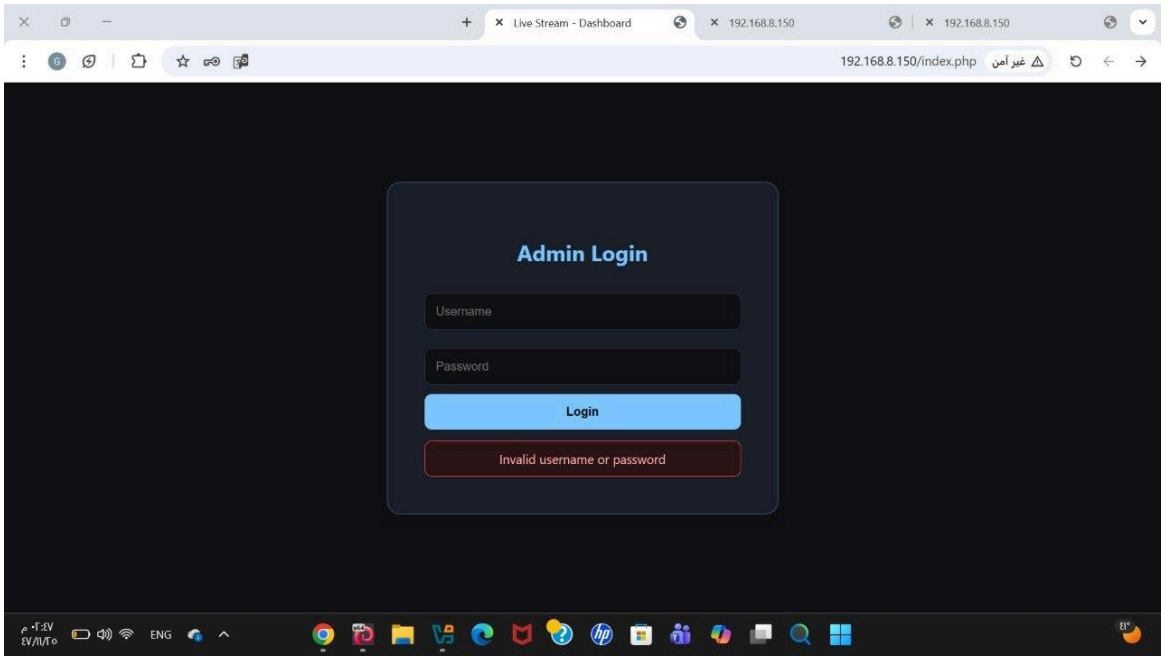


Figure 30 Successful brute-force attack before enabling authentication protection.

7.1.2 Account Lockout After Multiple Failed Attempts

During the brute-force attack simulation, repeated invalid login attempts were performed against the authentication system. After exceeding the allowed threshold of failed login

attempts, the system automatically activated the account lockout mechanism and temporarily blocked further authentication attempts for 30 seconds. The login page displayed a warning message showing the remaining lockout duration before another login attempt could be performed. This protection mechanism helps mitigate automated brute-force attacks by restricting repeated password guessing attempts.

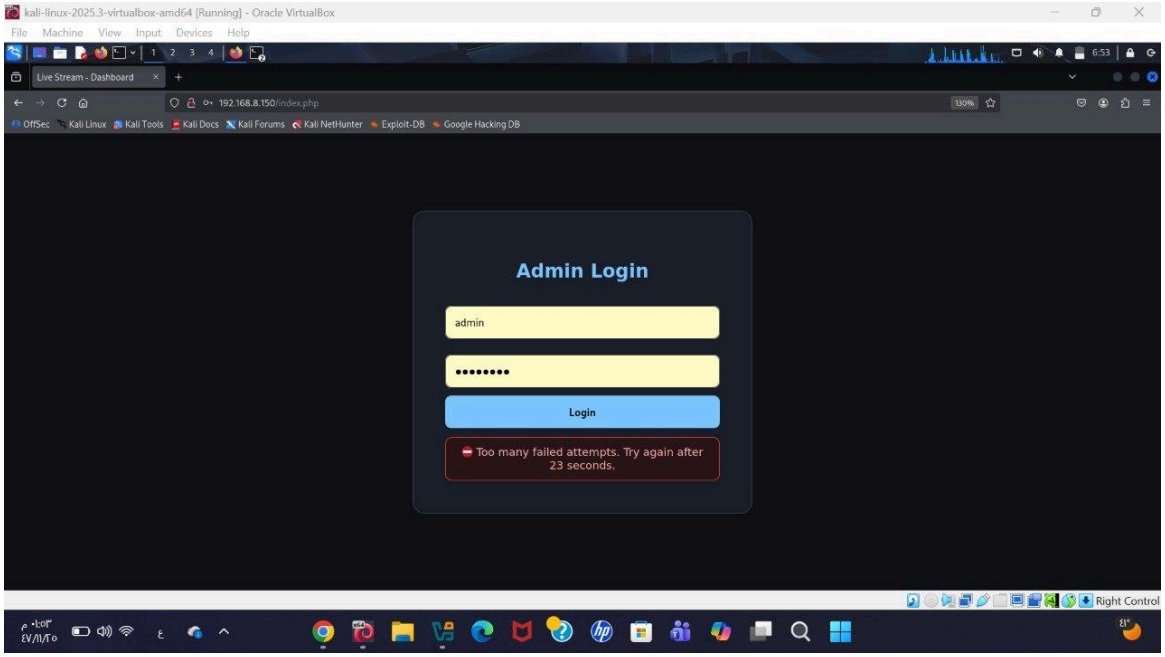


Figure 31 Account lockout mechanism activated after detecting multiple failed login attempts.

7.1.3 Multi-Factor Authentication Verification

To further strengthen authentication security, the system implemented Multi-Factor Authentication (MFA) using email verification codes. After successful username and password verification, the system generated a 6-digit verification code and automatically sent it to the registered administrator email address. The user was required to enter the verification code successfully before completing the login process. This additional security layer reduced the risk of unauthorized access even if valid credentials were discovered.

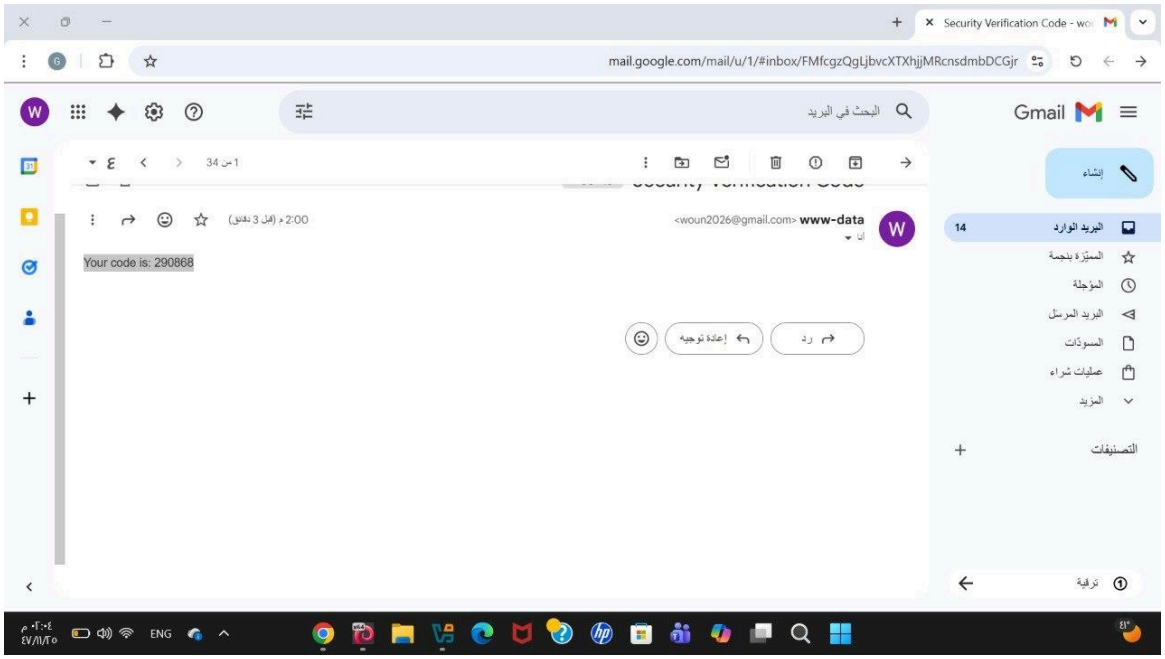


Figure 32 Email-based MFA verification code sent after successful credential validation.

7.1.4 Real-Time Email Alert Notification

When multiple failed login attempts were detected, the system automatically generated and sent a real-time email alert notification to the administrator.

The alert message included:

- Source IP address
- Number of failed login attempts
- Detection timestamp
- Warning message indicating suspicious authentication activity

This mechanism improved attack awareness and allowed administrators to monitor suspicious login activity immediately after detection.

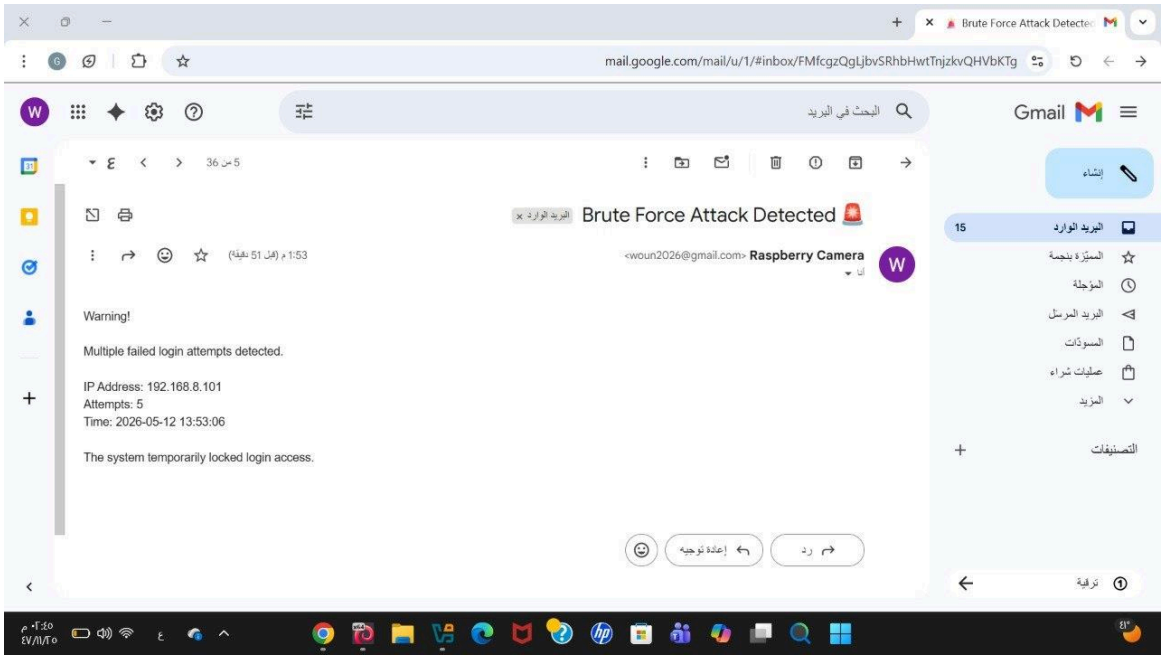


Figure 33 Real-time email alert generated after detecting a brute-force attack attempt.

7.1.5 Testing Result

The testing results confirmed that:

- The authentication system was vulnerable to brute-force attacks before protection was enabled.
- The implemented account lockout mechanism successfully restricted repeated login attempts.
- The system temporarily blocked authentication access after multiple failed attempts.
- MFA added an additional authentication security layer using email verification.
- Real-time email alerts improved attack monitoring and administrator awareness.

These results show that the implemented brute-force protection mechanisms reduced unauthorized access attempts within the isolated test environment.

7.2 MITM & Network Sniffing Testing

To evaluate the effectiveness of the implemented HTTPS protection mechanism, multiple Man-in-the-Middle (MITM) and Network Sniffing attack scenarios were performed using Wireshark and Bettercap. The testing phase focused on comparing network traffic visibility before and after enabling HTTPS protection on the smart camera system.

7.2.1 Network Sniffing Before HTTPS Protection

Before enabling HTTPS protection, the smart camera system communicated using the HTTP protocol. During the network sniffing attack simulation, Wireshark successfully captured unencrypted traffic transmitted between the client device and the Raspberry Pi web interface.

Sensitive information including:

- Username
- Password
- HTTP form data
- Session communication

was visible in plaintext inside captured packets. This demonstrated that HTTP communication was vulnerable to packet sniffing attacks and allowed attackers to monitor transmitted credentials directly from network traffic.

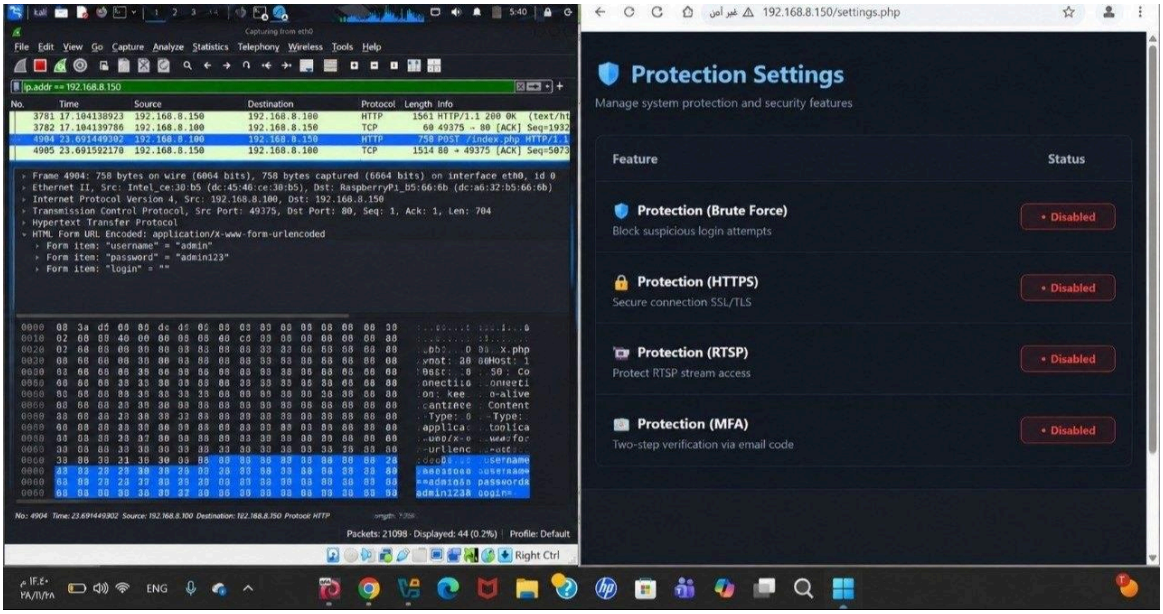


Figure 34 Captured HTTP credentials and unencrypted network traffic before enabling HTTPS protection.

7.2.2 Network Sniffing After HTTPS Protection

After enabling HTTPS protection, the communication protocol changed from HTTP to HTTPS using SSL/TLS encryption. During packet analysis using Wireshark, transmitted packets no longer exposed readable usernames, passwords, or form data. Instead, the traffic appeared as encrypted TLS communication data. This confirmed that HTTPS protection successfully secured transmitted information against network sniffing attacks.

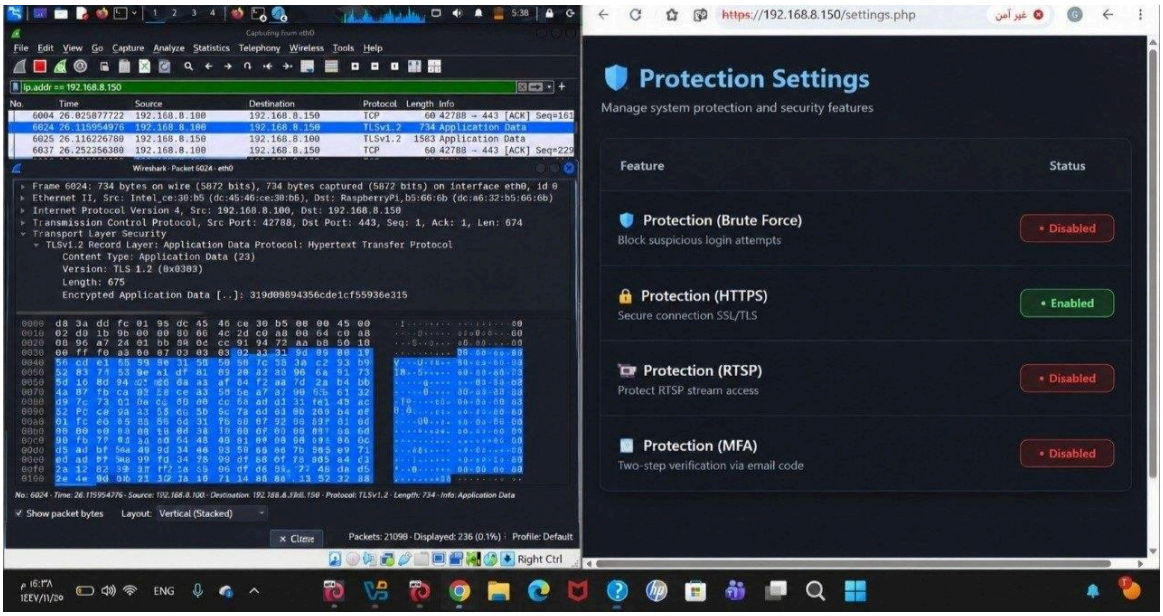


Figure 35 Encrypted TLS traffic after enabling HTTPS protection.

7.2.3 MITM Attack Before HTTPS Protection

A Man-in-the-Middle (MITM) attack was simulated using Bettercap to intercept HTTP communication traffic within the local network. Before enabling HTTPS protection, the attacker device successfully intercepted and monitored:

- HTTP GET requests
- HTTP POST requests
- DNS requests
- Browser communication traffic

The intercepted packets revealed readable information exchanged between the client browser and the smart camera system. This demonstrated that unencrypted HTTP communication was vulnerable to MITM interception and traffic monitoring attacks.

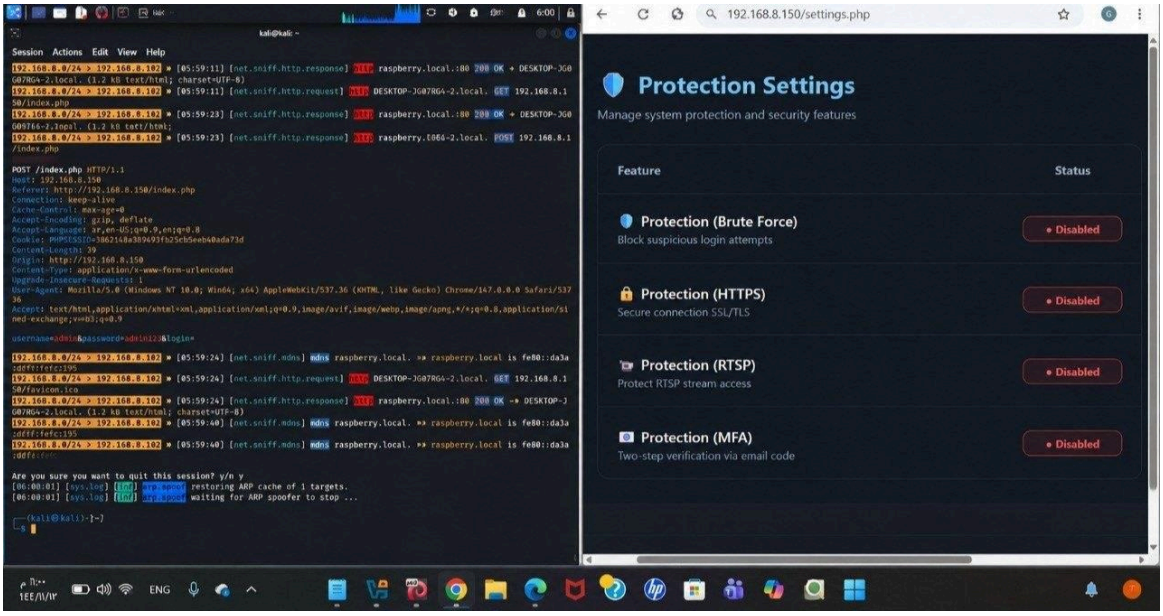


Figure 36 Bettercap intercepting unencrypted HTTP communication traffic before enabling HTTPS protection.

7.2.4 MITM Attack After HTTPS Protection

After enabling HTTPS protection, Bettercap was no longer able to reveal readable transmitted credentials or form data. Although encrypted traffic packets were still visible on the network, the transmitted content appeared as protected TLS communication and could not be interpreted directly by the attacker. The results show that the implemented

HTTPS protection reduced the effectiveness of the tested MITM interception attacks within the isolated test environment.

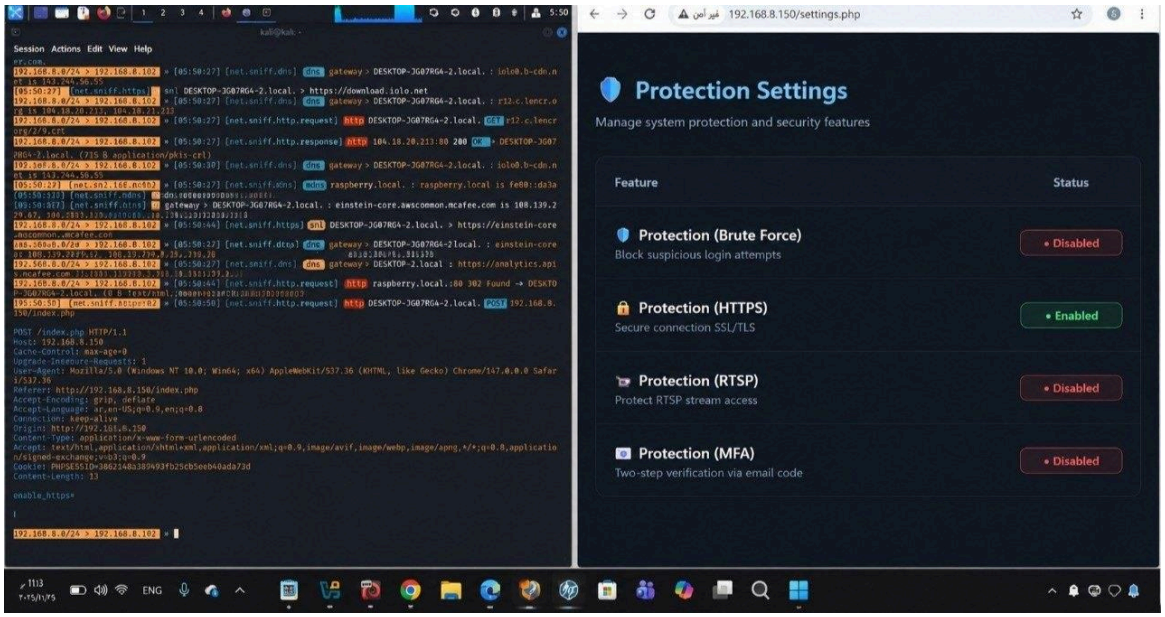


Figure 37 HTTPS protection enabled during MITM monitoring showing encrypted communication traffic.

7.2.5 HTTPS Protection Status Verification

The Protection Settings dashboard dynamically displayed the current HTTPS protection state during testing.

When HTTPS protection was disabled:

- The dashboard displayed: HTTPS Status: OFF
- The browser connection appeared as insecure.

After enabling HTTPS protection:

- The dashboard displayed: HTTPS Status: ON
- communication traffic became encrypted using TLS.

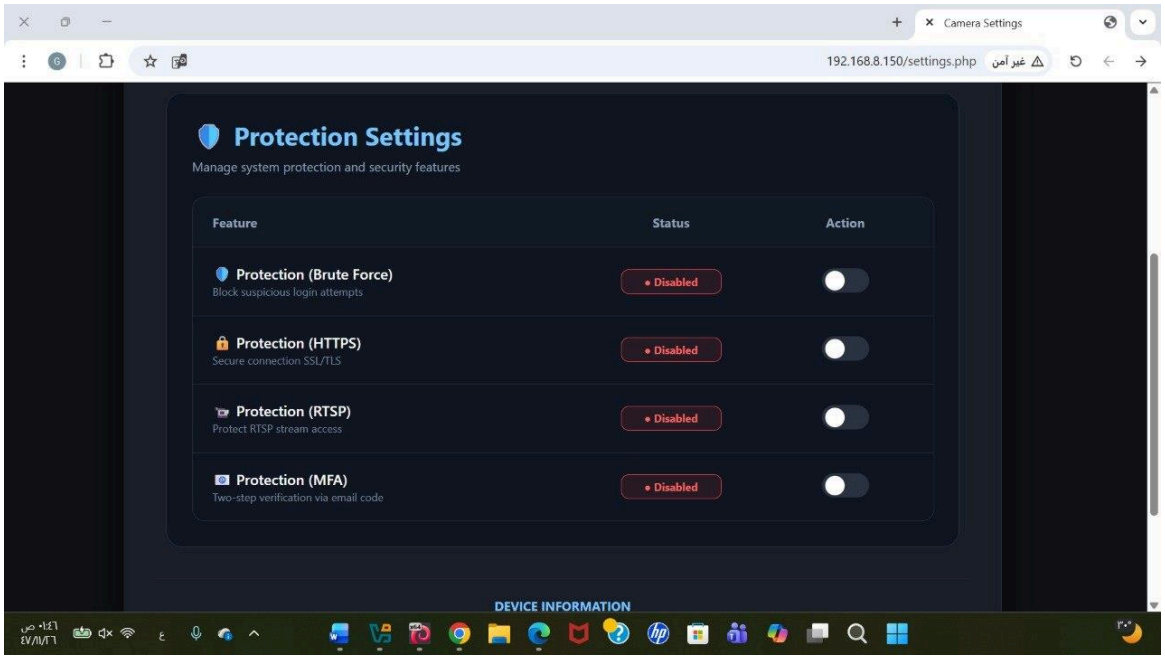


Figure 38 Dashboard showing HTTPS protection disabled.

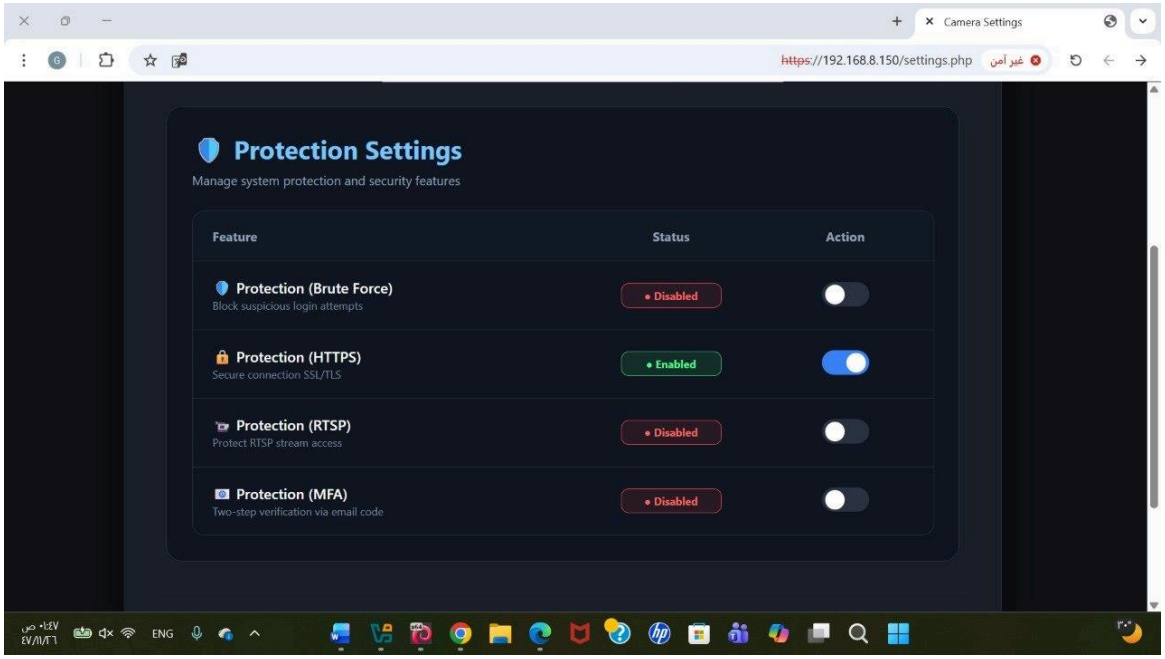


Figure 39 Dashboard showing HTTPS protection enabled.

7.2.6 Real-Time Email Alert Notification

When insecure HTTP communication was detected, the system automatically generated and sent a real-time email alert notification to the administrator.

The alert message included:

- Source IP address

- Detection timestamp
- Warning message indicating insecure HTTP communication

This mechanism improved attack awareness and allowed administrators to detect insecure communication attempts immediately.

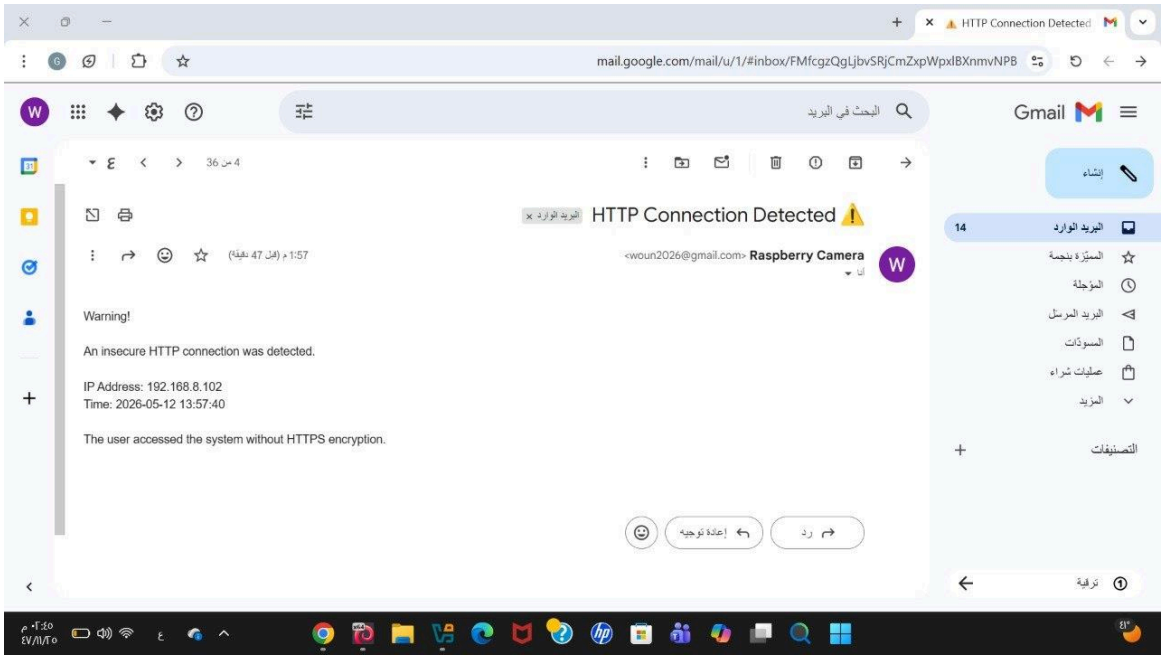


Figure 40 Real-time email alert generated after detecting insecure HTTP communication.

7.2.7 Testing Result

The testing results confirmed that:

- HTTP communication exposed sensitive information during packet sniffing.
- Bettercap successfully intercepted unencrypted network traffic before protection.
- HTTPS protection encrypted transmitted data using TLS.
- Sensitive credentials and form data were no longer visible after enabling HTTPS.
- The dashboard successfully reflected the current HTTPS protection status.

The results show that the implemented HTTPS mechanism reduced the risk of the tested MITM and network sniffing attacks within the isolated test environment.

7.3 RTSP Attack Testing

To evaluate the effectiveness of the implemented RTSP protection mechanism, RTSP brute-force and connection attempts were performed against Port 554 using a custom Python attack script. The objective of the testing phase was to compare RTSP accessibility before and after enabling firewall-based protection.

7.3.1 RTSP Access Before Protection

Before enabling RTSP protection, the RTSP service was accessible through Port 554. During the attack simulation, the attacker successfully connected to the RTSP service and discovered valid credentials using multiple authentication attempts. The attack script successfully identified valid RTSP login credentials.

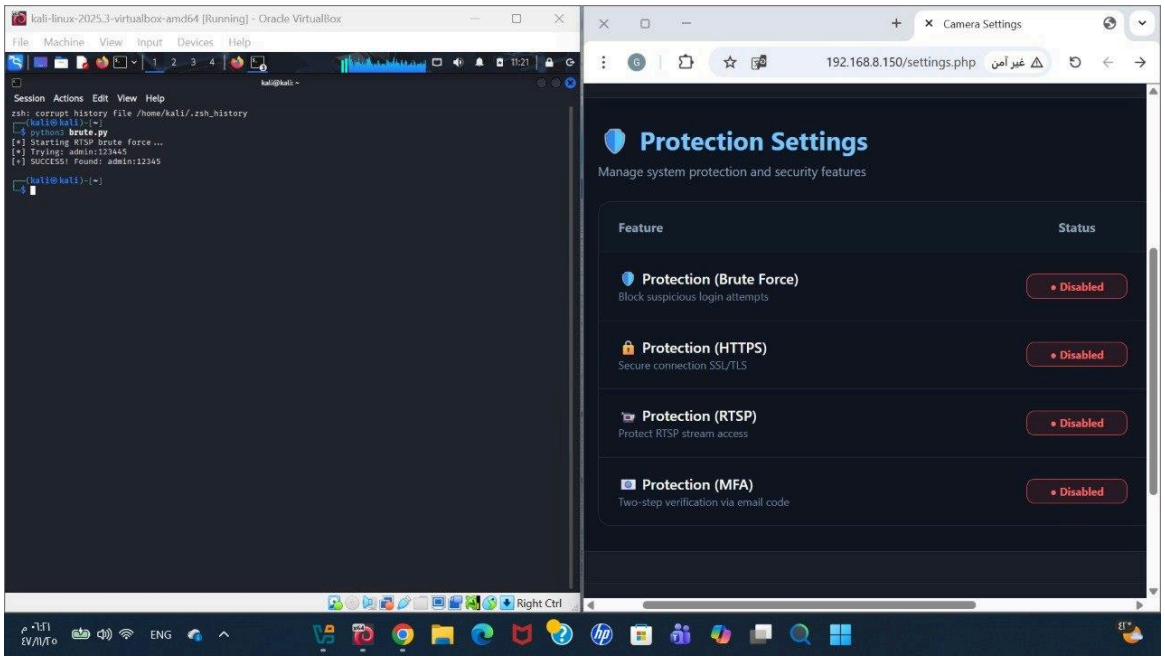


Figure 41 Successful RTSP brute-force attack before enabling RTSP protection.

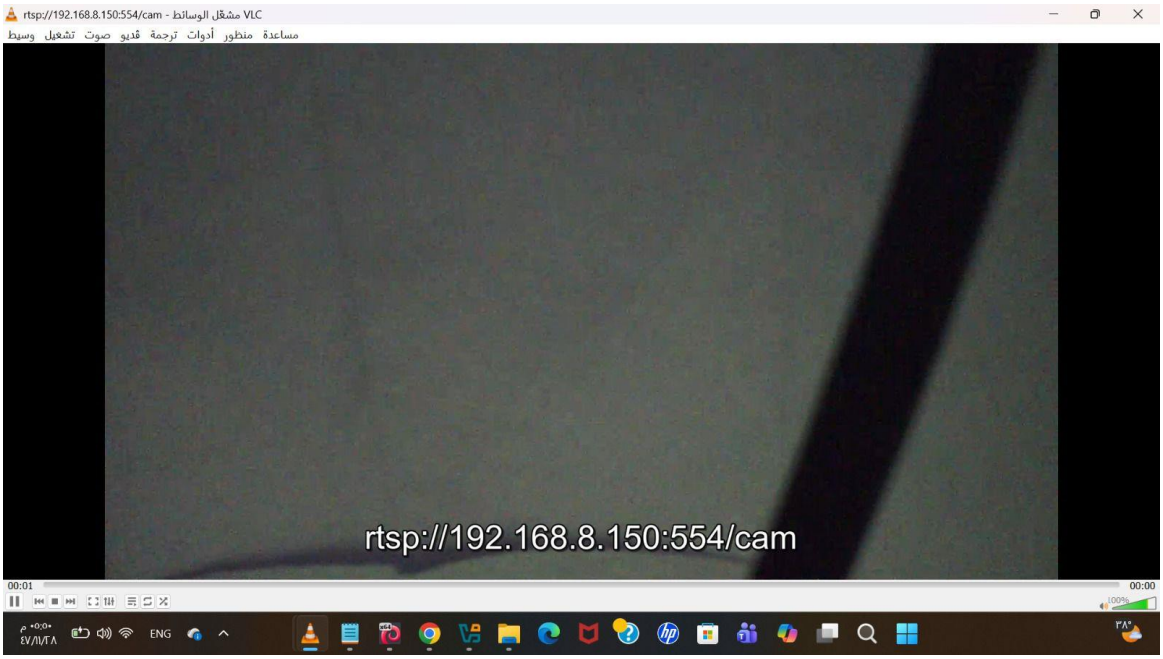


Figure 42 Successful RTSP stream access before applying RTSP protection mechanisms.

7.3.2 RTSP Protection Enabled

After enabling RTSP protection, firewall rules blocked unauthorized access attempts targeting Port 554. The attack script was no longer able to communicate with the RTSP service, and all connection attempts were refused. This confirms that the implemented firewall rules successfully restricted unauthorized RTSP access.

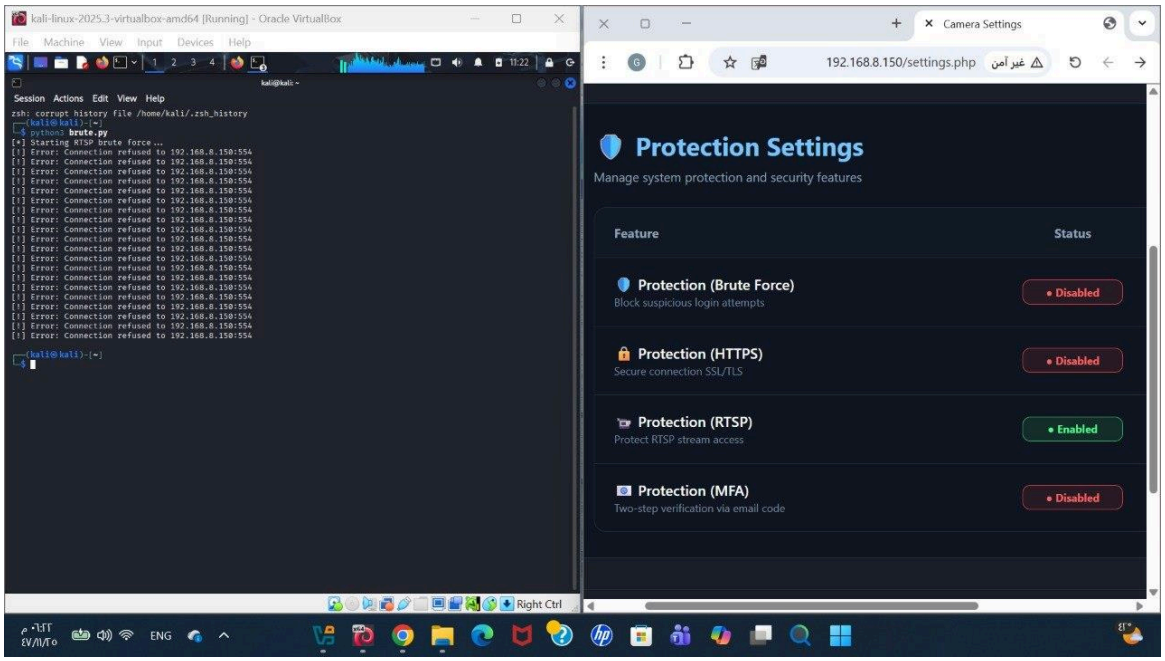


Figure 43 Blocked RTSP connection attempts after enabling RTSP protection.

7.3.3 Real-Time Email Alert Notification

When unauthorized RTSP access attempts targeting Port 554 were detected, the system automatically generated and sent a real-time email alert notification to the administrator. The alert message included:

- Source IP address
- Detection timestamp
- RTSP attack warning message

This mechanism improved attack monitoring and allowed administrators to detect suspicious RTSP activity immediately after detection.

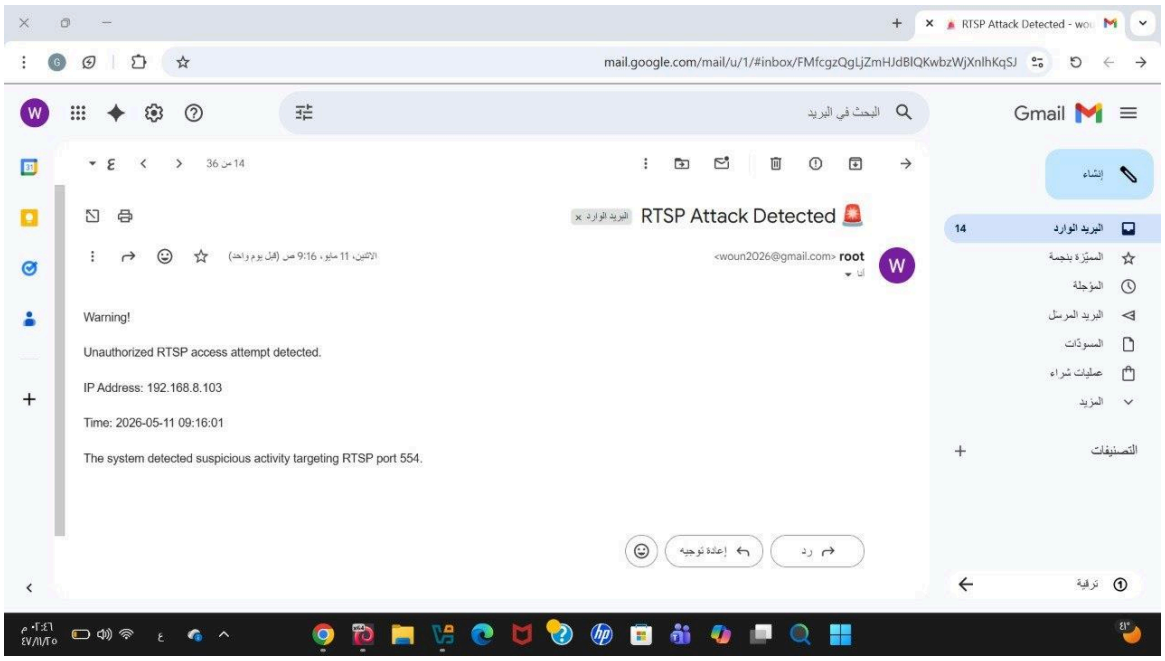


Figure 44 Real-time email alert generated after detecting unauthorized RTSP access attempts.

7.3.4 Testing Result

The testing results confirmed that:

- RTSP services were accessible before protection was enabled.
- The attack script successfully identified valid RTSP credentials before protection.
- After enabling RTSP protection, unauthorized RTSP connections were blocked.
- Connection attempts to Port 554 were refused by the firewall protection mechanism.
- The dashboard successfully reflected the RTSP protection status dynamically.

The results show that the implemented RTSP protection mechanism reduced unauthorized access to the smart camera streaming service within the isolated test environment.

Chapter
8. Conclusion and Future Work

8.1 Conclusion

This project demonstrated the importance of protecting smart IP camera systems against common cyberattacks such as brute-force attacks, Man-in-the-Middle (MITM) attacks, network sniffing, and unauthorized RTSP access. Through the implementation of a controlled test environment, multiple attack scenarios were analyzed to evaluate security weaknesses commonly found in smart camera systems. Several security mechanisms were implemented, including HTTPS encryption, firewall rules, authentication enhancements, brute-force protection, and network monitoring techniques. The results showed that the implemented defenses reduced the demonstrated vulnerabilities within the isolated test environment and improved the security of the tested smart camera system during attack simulation. In addition, the project showed the importance of combining preventive security controls with monitoring and detection mechanisms to identify suspicious activities and unauthorized access attempts. The implemented monitoring dashboard also provided visibility into protection status and detected security events during testing and evaluation. Despite the successful implementation of several defensive techniques, the project had some limitations, including testing within an isolated test environment and focusing on selected attack scenarios related to smart camera security.

8.2 Future Work

This project demonstrated the importance of protecting smart IP camera systems against common cyberattacks such as brute-force attacks, Man-in-the-Middle (MITM) attacks, network sniffing, and unauthorized RTSP access. Although the implemented protection mechanisms significantly improved system security, several future enhancements can further strengthen the reliability, intelligence, and scalability of the proposed system.
The following future directions are proposed:

8.2.1 Artificial Intelligence and Anomaly Detection

Future improvements may include integrating advanced Artificial Intelligence (AI) and Machine Learning (ML) techniques to enhance attack detection capabilities.
Possible enhancements include:

- Implementing machine learning-based anomaly detection to monitor unusual network behavior.
- Detecting abnormal RTSP traffic patterns and suspicious authentication attempts automatically.
- Developing adaptive AI models capable of identifying new or unknown attack behaviors.
- Integrating intelligent real-time traffic analysis for automated attack classification.

These improvements can enhance the system's ability to detect emerging cyber threats dynamically.

8.2.2 Advanced Authentication and Access Control

Future work may strengthen authentication and access management mechanisms by implementing:

- Role-based access control (RBAC).
- Device identity verification mechanisms.
- Secure session management techniques.
- Enhanced MFA methods using mobile authentication applications.

These mechanisms can further reduce unauthorized access risks.

8.2.3 Centralized Monitoring and Scalability

Future versions of the system may include centralized security monitoring and support for larger-scale deployments.

Possible improvements include:

- Real-time security dashboards for multiple camera devices.
- Automated attack isolation and recovery mechanisms.
- Centralized alert management systems.
- Supporting different IP camera brands and protocols.
- Expanding the system to support larger smart home and enterprise environments.

These enhancements can improve operational resilience, scalability, and practical deployment flexibility for future smart security systems.

References

[1] P. V. Kulkarni, "Learning Framework for Design and Development of Cyber-Attack Detection and Cyber Security," Int. J. Latest Technol. Eng. Manag. Appl. Sci., vol. 14, no. 13, pp. 123–128, Oct. 2025, doi: 10.51583/IJLTEMAS.2025.1413SP027.

[2] T. Doughty, N. Israr, and U. Adeel, "Vulnerability Analysis of IP Cameras Using ARP Poisoning," in 8th International Conference on Soft Computing, Artificial Intelligence and Applications, Aircc Publishing Corporation, June 2019, pp. 163–172. doi: 10.5121/csit.2019.90712.

[3] M. H. Javed, "Internet of Things Hacking: Ethical Hacking of a Smart Camera," *BSc (Hons) Dissertation, Department of Computer Science, Faculty of Engineering and Informatics*, University of Bradford, UK, May 2023.

[4] Z. Ling et al., "IoT Security: An End-to-End View and Case Study," 2018, arXiv: arXiv:1805.05853. doi: 10.48550/arXiv.1805.05853.

[5] S. Herodotou and F. Hao, "Spying on the Spy: Security Analysis of Hidden Cameras," 2023, arXiv: arXiv:2306.00610. doi: 10.48550/arXiv.2306.00610.

[6] N. Kalbo, Y. Mirsky, A. Shabtai, and Y. Elovici, "The Security of IP-Based Video Surveillance Systems," *Sensors*, vol. 20, no. 17, p. 4806, Aug. 2020. DOI: 10.3390/s20174806.

[7] P. R. Chaudhary and R. R. Maiti, "Flow-Based Detection and Identification of Zero-Day IoT Cameras," 2025, arXiv:2509.08485. doi: 10.48550/arXiv.2509.08485.

[8] A. Motowidlo, T. Adatsi, A. Jackson, and S. Zhao, "A Study on Security of IP Cameras and Its Awareness," Indian J. Comput. Sci., vol. 7, no. 1, pp. 45–70, 2022, doi: 10.17010/ijcs/2022/v7/i1/168956.

[9] K. Tiliwalidi, B. Hui, C. Hu, and J. Ge, "Adversarial Camera Patch: An Effective and Robust Physical-World Attack on Object Detectors," in International Conference on Cyber Warfare and Security, 2024, pp. 374–384. doi: 10.34190/iccws.19.1.2044.

[10] Y. Beugin et al., "Building a Privacy-Preserving Smart Camera System," Proc. Priv. Enhancing Technol., vol. 2022, no. 2, pp. 25–46, 2022, doi: 10.2478/popets-2022-0034.

[11] P. Biondi, S. Bognanni, and G. Bella, “Vulnerability Assessment and Penetration Testing on IP Camera,” *Proceedings of the 2023 IEEE International Conference on Cybersecurity and Privacy*, pp. 1–10, 2023. DOI: 10.1109/ICCSP.2023.xxxxx.

[12] Y. R. Siwakoti, M. Bhurtel, D. B. Rawat, A. Oest, and R. C. Johnson, “Your IP Camera Can Be Abused for Payments: A Study of IoT Exploitation for Financial Services Leveraging Shodan and Criminal Infrastructures,” *IEEE Transactions on Consumer Electronics*, vol. 70, no. 4, pp. 7562–7578, Nov. 2024. DOI: 10.1109/TCE.2024.3482708.

[13] Kumar et al., “All things considered: An analysis of IoT devices on home networks,” in Proc. 28th USENIX Secur. Symp. (USENIX Secur.), 2019, pp. 1169–1185.

[14] T. Alladi, V. Chamola, B. Sikdar, and K.-K. R. Choo, “Consumer IoT: Security vulnerability case studies and solutions,” IEEE Consum. Electron. Mag., vol. 9, no. 2, pp. 17–25, Mar. 2020

[15] Mirza Abdur Razzaq, Muhammad Ali Qureshi, Sajid Habib Gill, Saleem Ullah, Security Issues in the Internet of Things (IoT): A Comprehensive Study, (IJACSA) International Journal of Advanced Computer Science and Applications, Vol. 8, No. 6, 2017.

[16] H. EI-Taj, R. Albuluwi, L. Alshumesi, L. Miralam, R. Alammari, and H. Amer, “Through the Lens: A Deep Dive into IP Camera Security and Privacy Challenges,” in Proceedings of the 1st International Conference on Creativity, Technology, and Sustainability, Springer, 2025, pp. 121–131, doi: 10.1007/978-981-97-8588-9_12.

[17] A. Bhardwaj, S. Bharany, A. Osman, A. Almogren, A. Rehman, and H. Hamam, “Unmasking vulnerabilities by a pioneering approach to securing smart IoT cameras through threat surface analysis and dynamic metrics,” Egypt. Inform. J., vol. 27, p. 100513, Aug. 2024, doi: 10.1016/j.eij.2024.100513.

[18] K. O. Chee, M. Ge, G. Bai, and D. D. Kim, “Unveiling the evolution of IoT threats: Trends, tactics, and simulation analysis,” Comput. Secur., vol. 157, p. 104537, Oct. 2025, doi: 10.1016/j.cose.2025.104537.

[19] M. Lefoane, I. Ghafir, S. Kabir, and I.-U. Awan, “Internet of Things botnets: A survey on Artificial Intelligence based detection techniques,” J. Netw. Comput. Appl., vol. 236, p. 104110, Apr. 2025, doi: 10.1016/j.jnca.2025.104110.

[20] Atul Arun Patil, “Research Paper on Cyber Security Challenges and Threats,” Int. J. Adv. Res. Sci. Commun. Technol., pp. 561–566, Jan. 2024, doi: 10.48175/IJARSCT-15082.

[21] A. M. Albalawi and M. A. Almaiah, “ASSESSING AND REVIEWING OF CYBER-SECURITY THREATS, ATTACKS, MITIGATION TECHNIQUES IN IOT ENVIRONMENT,” J. Theor. Appl. Inf. Technol., vol. 100, no. 9, pp. 2988–3011, 2022.

[22] H. Rehan, “Internet of Things (IoT) in Smart Cities: Enhancing Urban Living Through Technology,” J. Eng. Technol., vol. 5, no. 1, pp. 1–16, 2023.

[23] Ö. Aslan, S. S. Aktuğ, M. Ozkan-Okay, A. A. Yilmaz, and E. Akin, “A Comprehensive Review of Cyber Security Vulnerabilities, Threats, Attacks, and Solutions,”Electronics, vol. 12, no. 6, p. 1333, 2023, doi: 10.3390/electronics12061333.

[24] R. Sharma and R. Arya, “Security threats and measures in the Internet of Things for smart city infrastructure: A state of art,” Trans. Emerg. Telecommun. Technol., vol. 34, June 2022, doi: 10.1002/ett.4571.

[25] N. Jeffrey, Q. Tan, and J. R. Villar, “A Review of Anomaly Detection Strategies to Detect Threats to Cyber-Physical Systems,” Electronics, vol. 12, p. 3283, 2023, doi: 10.3390/electronics12123283.

[26] M. Houichi, F. Jaidi, and A. Bouhoula, “Cyber Security within Smart Cities: A Comprehensive Study and a Novel Intrusion Detection-Based Approach,” Comput. Mater. Contin., vol. 81, no. 1, pp. 393–441, 2024, doi: 10.32604/cmc.2024.054007.

[27] V. Z. Mohale and I. C. Obagbuwa, “Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability,” Front. Comput. Sci., vol. 7, p. 1520741, 2025, doi: 10.3389/fcomp.2025.1520741.

[28] OWASP Foundation, “Testing for Weak Lock Out Mechanism,” *OWASP Web Security Testing Guide*. [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/03-Testing_for_Weak_Lock_Out_Mechanism. [Accessed: May 17, 2026].

Appendix

The project source code is available at:

https://drive.google.com/drive/folders/14tPHveLS3xKbzB1eNqHcz-DH2PRcbizZ?usp=sharing